Knowledge-Driven Intent Life-Cycle Management for Cellular Networks

Kashif Mehmood, *Graduate Student Member, IEEE*, Katina Kralevska, *Member, IEEE*, and David Palma

***Abstract*—The management of cellular networks and services has evolved due to the rapidly changing demands and complexity of service modeling and management. This paper uses intent-based networking (IBN) as a solution and couples it with contextual information from knowledge graphs (KGs) of network and service components to achieve the objective of service orchestration in cellular networks. Fusing IBN with KGs facilitates an intelligent, flexible, and resilient service orchestration process. We propose an intent completion approach using knowledge graph learning and a mapping model capable of inferring and validating the service intents in the network. Subsequently, these service intents are deployed using available network resources in a simulated fifth generation (5G) non-standalone (NSA) network. The compliance of the deployed intents is monitored, and mutual optimization against their required service key performance indicators is performed using Simultaneous Perturbation Stochastic Approximation (SPSA) and Multiple Gradient Descent Algorithm (MGDA). The numerical results show that the knowledge graph with Gaussian embedding (KG2E) model outperforms other distance-based embedding models for the proposed service KG. Different combinations of strict latency (SL) and non-strict latency (NSL) intents are deployed, and compliance is evaluated for increasing numbers of deployed intents against baseline deployment scenarios. The results show a higher level of compliance for SL intents to target latencies in comparison to NSL intents for the proposed intent deployment and optimization algorithm.**

***Index Terms*—IBN, closed-loop control, service model, knowledge graph learning, service and network management, optimization.**

I. Introduction

THE RAPID advancement of networking technologies has rendered traditional management paradigms insufficient for addressing the needs of modern, sophisticated, and dynamic network environments. As networks grow in scale and complexity, controlling such systems becomes increasingly challenging, necessitating more intelligent and adaptable solutions. Intent-based networking (IBN) emerges as a promising approach to address these challenges, transitioning network administration from a manual, error-prone process to automated, policy-driven frameworks. Central to IBN is the concept of *intent* – a high-level, human-readable declaration of operational goals.

Received 9 October 2024; revised 15 April 2025; accepted 9 June 2025. Date of publication 16 June 2025; date of current version 7 October 2025. The associate editor coordinating the review of this article and approving it for publication was R. Riggio. *(Corresponding author: Kashif Mehmood.)*

The authors are with the Department of Information Security and Communication Technology, Norwegian University of Science and Technology, 7491 Trondheim, Norway (e-mail: kashif.mehmood@ntnu.no; katinak@ntnu.no; david.palma@ntnu.no).

Digital Object Identifier 10.1109/TNSM.2025.3579547

The TeleManagement Forum (TM Forum) defines intent as a *"formal specification of all expectations including requirements, goals, and constraints"* [1]. In cellular networks, IBN enables network stakeholders (e.g., network administrators, service and infrastructure providers, and the service consumers) to collaboratively design and deploy services [2]. These stakeholders specify network and service delivery objectives as abstract statements, allowing intent providers to design and implement deployment policies for suitable cellular network functions and services. However, translating these intents into actionable configurations across diverse network domains, while balancing competing stakeholder requirements, remains a significant challenge.

There are several challenges in understanding and processing intents in the networking domain with the utilization of available information from different network stakeholders [3], [4], [5]. Abstraction in intent requires a deeper understanding of the underlying network state, the network components' capabilities, and the network users' expectations. The dynamic nature of modern network environments further complicates the translation and mapping of intents to specific network and service concepts and configurations. Moreover, a compliant deployment of intents in a dynamic network environment requires optimizing the existing intents in the system as per their translated expectations [6], [7] and changing network dynamics.

Intent completion – augmenting high-level intent descriptions with contextual information addresses these challenges by leveraging diverse data sources (e.g., domain knowledge, historical service lifecycle data). Knowledge organization plays a critical role. Knowledge graphs (KGs) and ontologies have been widely used in computer science and the World Wide Web to organize different types of domain information with varying levels of hierarchy and relations. This is our primary motivation behind utilizing a knowledge-centric approach to understand the contents of a high-level intent and relate them to an application use case such as a service orchestration domain in fifth generation (5G) networks.

The Web Ontology Language (OWL) [8] provides the necessary hierarchical information integration with KGs, which efficiently represents processed and understood information from heterogeneous sources in a system. The Resource Description Framework (RDF) [9] provides a description format for data stored in a KG with extensive use cases in the context of graph theory and knowledge representation. A simple graph consists of a *subject (s), object (o),* and *predicate (p)*, where the subject and object nodes are

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

connected through the predicate. For example, *"IntentA has an expectation Exp1"* is represented as RDF triple $(s, p, o) = (IntentA, hasExpectation, Exp1)$. In this way, any data object can be represented using a KG in the form of RDF triples.

Intents are essentially KGs with abstract information regarding expected outcomes, such as service deployment. Using inference, relevant information can be extracted from the intent to design specific network objectives and policies. KGs support reasoning to uncover latent relationships and features of the available information for multiple interpretations of the data. These features are embedded as information in knowledge graphs, enabling the translation of ambiguities during intent translation and validation before deployment. For example, TM Forum [10] provides a blueprint ontology for intents and associated concepts for autonomous network management. However, there is a lack of literature on the fusion between KGs and their role in intent processing, translation, and validation. We focus on the problem of intent completion and compliant deployment using knowledge graph learning and multi-objective optimization.

In our previous work [6], we designed a cellular service orchestration extension model using a knowledge graph and ontology primitives. We utilized the presented service orchestration knowledge graph from [6] in our initial investigation of knowledge graph embedding in [11]. The work in [11] describes a basic model for knowledge embedding and its possible application in IBN systems. However, we do not formulate the system model or perform a comparative analysis with different knowledge graph embedding (KGE) models. Moreover, the explanation regarding the role of KGE is also lacking in [11].

This paper extends our previous work [11] by completing the intent processing pipeline with a deployment closed-loop and the intent translation closed-loop by utilizing the service orchestration extension model from [6]. The focus is on the comparative analysis of different KGE models and the development of an intent deployment, assurance, and monitoring framework. To this end, we seek to bridge the gap between the abstraction in intent and the specific requirements needed for cellular network operation, facilitating a more compliant, efficient, and user-aligned deployment of available resources. From a service orchestration perspective, we investigate the use of KGs in the intent lifecycle [2] from expression, translation, and mapping to their deployment. Moreover, a closed-loop control mechanism is designed to dynamically adapt the deployed intent's objectives to the network state using TM Forum's intent ontology [10] as a blueprint for the design and scope of intents.

Consequently, the contributions of this paper are listed below:

- *Intent service orchestration workflow:* We define an IBN-inspired service orchestration workflow for cellular networks to augment intent lifecycle processes with a typical service orchestration model. This workflow segregates different intent stakeholders and their roles in provisioning mapped services to the user intent generated by the communication service customer (CSC). The proposed workflow also highlights the role of knowledge-based intent translation and service predictions.
- *Knowledge representation model for service triple prediction and intent completion:* We explore knowledge graph embedding techniques, defining intents as RDF triples and using their projections in Gaussian space for predictions during the intent processing and translation. The model captures the existing relations between different concepts in the domain of service orchestration in cellular networks.
- *Active intent deployment and optimization:* Throughout their lifecycle, intents are verified against network state dynamics (resources and services) and adjusted accordingly. This is accomplished using a combination of Simultaneous Perturbation Stochastic Approximation (SPSA) [12] and Multiple Gradient Descent Algorithm (MGDA) [13] algorithms to achieve the expected Quality-of-Service (QoS) parameters in a multi-intent scenario.
- *Performance evaluation of IBN orchestrated services:* We have developed a 5G non-standalone (NSA) implementation to validate the proposed intent-based service orchestration framework results. This setup consists of two modules - an intent and KG processing module and an implementation and optimization module. Experiments are performed with different service combinations valid in 5G networks to measure the performance of the proposed intent processing pipeline and optimization model for deploying orchestrated services. For each service orchestrated by its respective intent, the difference between measured and expected performance is evaluated to solve the proposed multi-objective optimization problem.

The rest of the paper is organized as follows. We present relevant works from the literature in Section II, while Section III formulates the system model for the intents and their mapping relationships with services and QoS parameters. In addition, it also presents the proposed knowledge graph embedding model for intent translation and validation. Section IV proposes the closed-loop optimization model for the translated intents using SPSA and MGDA algorithms. Section V establishes the experimental setup and verification of the proposed methods for the intent translation and mapping components. Section VI analyzes the optimal deployment framework for the translated intents as services with different experimental scenarios. Finally, Section VII concludes the paper by discussing prospective challenges and future directions.

II. Related Works

This section presents state-of-the-art progress regarding IBN, the inherent role of knowledge and context in network management, and available works utilizing KGs for intent translation and closed-loop (CL) control in the intent lifecycle.

A. IBN Management

IBN offers diverse features to enhance the management of traditional communication networks through automated

workflows, adaptability to changing network dynamics, and alignment with high-level business objectives of different intent stakeholders. Recent studies have investigated the vision, challenges, and open issues related to IBN and migration paths from traditional networks [14], [15], [16], [17], [18], [19], [20], [21], [22], [23].

Yu et al. [14] present a comprehensive, open-source, and standards-based framework for IBN, discussing its architecture and implementation of key IBN aspects like intent lifecycle, translation, orchestration, and assurance. Clemm and Eckert [15] investigate the synergy between autonomous networks and IBN by reviewing the standardization efforts of the Internet Engineering Task Force (IETF) for IBN systems. They also identify the potential enhancements in IBN to achieve autonomy in its lifecycle management.

A survey by Leivadeas and Falkner [18] outlines the essential components for an autonomous IBN system, emphasizing the importance of each element from intent expression to assurance. It also identifies current technological gaps that hinder the realization of a fully autonomous network, thereby guiding future research directions. Zhang et al. [19] model an intent-driven CL control of network slices based on stringent QoS requirements for different services. A goal decomposition approach is adopted using event calculus, and Pyke-based logical reasoning over combined goals for various services. The resource allocation is managed through a deep Q-learning network (DQN), and the performance is validated through an open radio access network (RAN) implementation testbed.

A more practical approach is adopted by Lingga et al. [20], where they propose utilizing IBN and intelligent functionality in a futuristic sixth generation (6G) core network, integrated with existing 5G core functions. They propose an intelligent data monitoring solution built upon network analytics gathered by the network data analytics function (NWDAF) to introduce IBN management in the proposed framework. Moreover, Massa et al. [21] present a virtualized network function (VNF) configuration model using declarative high-level objective (aka intents) as inputs in the construction of a VNF chain.

In another study, Abassi and Tabbane [22] explore an intent lifecycle with an engineering approach focusing on the context awareness and role of artificial intelligence (AI) in different lifecycle stages. The proposed lifecycle analyzes available information sources in the networks to design and implement intent deployment and monitoring strategies in an adaptive and conflict-free way. Finally, Yang et al. [23] describe SMART - an intent-driven management architecture that promotes fully autonomous network management from system and user perspectives, ensuring measurable and achievable network management intents.

B. Knowledge Graphs in Networking

Knowledge organization and management are essential for maintaining an updated model of information sources in communication networks. Numerous studies explore the network management paradigm by applying knowledge and information extraction, utilization, and consumption methods [24], [25], [26], [27], [28].

Mier et al. [24] analyze the application of KGs in the management of telecommunication systems, emphasizing their role in integrating heterogeneous data sources and automating complex decisions. KGs improve resource and service discovery and monitoring, particularly within software-defined network (SDN), though challenges like data quality and graph performance must be addressed for broader adoption in telecommunication systems. Jie et al. [25] summarize the application of KG in various industries, enhancing decision support and intelligent systems. This review identifies the potential of KGs to revolutionize vertical applications by effectively organizing complex, heterogeneous information.

Mitropoulou et al. [26] leverage KG learning and link predictions to detect and mitigate network congestion scenarios in service delivery. A network infrastructure KG is created using the available network topology and connectivity information, enabling congestion prediction to initiate corrective mitigation actions. Similarly, He et al. [27] address the representation learning problem of KGs for wireless communication networks. They propose a comprehensive KG generation methodology using communication protocol and domain expert knowledge for 5G and beyond core network. They utilize graph neural network-based architecture to generate a learning model for the proposed KG. Mitropoulou et al. [28] target cloud resource orchestration by training KGE models to optimize resource allocation. They deploy unsupervised learning with KGE input to detect anomalies in cloud resource reservation and propose a re-optimization mechanism to resolve them.

C. Knowledge Graphs and IBN

We reviewed works focusing on the application of KG theory in IBN, given the critical role of context and network dynamics in the intent lifecycle [4], [6], [10], [29], [30], [31], [32], [33], [34], [35], [36]. TM Forum‘s intent management ontology describes the structural components in an intent [29]. This ontology defines the expected intent hierarchy concerning the ownership, dependencies, and lifecycle. Moreover, TM Forum also defines the intent common model (ICM) [10] for the definition and determination of the scope of the intent itself. The model includes classes such as *Expectation, Parameter,* and *Reporting,* defining the functional and non-functional requirements for the intent. Notably, this is the only publicly available ontology defining the specifics of the IBN. Our work adopts this ICM as the blueprint for designing an intent knowledge graph (IKG).

The intent language specification [10] is defined to aid the utilization of available information in the IKG. Inspired by the TM Forum‘s intent model and ontology, a KG with extension to the mission-critical and non-mission critical service definitions in 5G network specifications is proposed by Mehmood et al. [6]. The intent translation process utilizes SPARQL queries to retrieve information from a knowledge base containing service types, QoS parameters, and resource type, as intents tend to become domain-specific until they are ready to be deployed. Daroui et al. [30] explore the problem of maintaining a synchronized state of knowledge across the

entire IBN domain with the help of TM Forum's intent management model. The authors propose a state management framework to handle multiple states of the knowledge base that are beneficial for access to historical states relevant to the network's current and future states of knowledge. The proposed work arranges knowledge base instances in addition to the current state for agents' utilization to process intents.

Zhang et al. [31] handle conflict management problems in IBN by partially utilizing a KG to model network-level information. They propose that users follow a constrained grammar to model intents as a quadruple for grouped network nodes in a domain. The conflicts are resolved by producing chains of service functions necessary for intent deployment in the network according to the priority levels. Xiao et al. [32] utilize a simplistic KG for stored information regarding user intents based on expected user requirements and associated attributes. The intent translation process uses the stored information in the network KG to perform a secondary match with the stored intents for user inputs. An open network operating system (ONOS) intent is generated after the knowledge reconciliation is completed and handled by an ONOS controller. Similar work, with the ONOS framework, by Chang et al. [4] utilizes a hierarchical KG model with separate graphs for the intent layer and network layer. These KGs are used to reach a cooperative agreement while processing intents, matching users' requested features to the available resources in the physical network.

Wang et al. [33] consider a smart grid use case and propose a KG design scheme and an associated IBN framework. A data fusion mechanism is designed to construct KGs using data from various sources within the smart grid vertical, such as QoS requirements, network state, resource state, and automation state. However, it lacks validation and evaluation when using the proposed KG. Similarly, Guo et al. [34] recommend using graph neural networks as a basis for an automated generation of a configuration framework for autonomous networking using KGs. They utilize different known configuration templates as imperative intents to train the graph neural network to generate configuration templates for a metro network use case.

Kou et al. [35] propose SAFLA, a semantically aware lifecycle for IBN, with an objective of intent translation to fill the semantic gap between high-level intent and low-level configurations. The lifecycle consists of intent translation, conflict resolution, policy generation, and deployment with an ONOS controller, followed by intent assurance. The intent in different stages and the semantic information regarding network infrastructure are collected and represented as KGs. Furthermore, AMANOS [36], presented by Boutouchent et al., focuses on the lifecycle management of intents using a hierarchical intent propagation model based on the traditional network management layers [37]. Different KG components are organized as a knowledge base that consists of intent, network, and resource states along with designed intent deployment policies. The knowledge base is a critical reference point in the intent lifecycle and management decisions.

Sabour et al. [38] investigate the generation of service-level KGs by extracting the functional requirements from the user intents. They propose a domain ontology for intent translation and a catalog of service templates for KG generation. Ontologies are an important blueprint for organizing knowledge in IBN as depicted in the following works [39], [40], [41]. Ouyang et al. [40] translate the user's natural language intent by building an intent ontology. The intent processing uses the intent tuples and network configuration protocol (NETCONF) templates. Moreover, the intent refinement process consists of natural language processing (NLP) and semantic-level verification for the user intent and intent tuples before low-level configurations. Randles et al. [39] model an intent-based control loop ontology (IBCLO) for intent representation using time-series metrics from monitored services. They investigate a 5G slicing use case to verify the proposed RDF graph generator (PRG) and the IBCLO for provisioning QoS guarantees to the deployed intents as different slices in the network. Dragun and Bogunović [41] propose a knowledge-driven approach using a knowledge structure known as granules, similar to KGs, to reason and drive intent-managed systems. They use reusable knowledge granules to model application use cases and their associated functional requirements. This reuse simplifies knowledge management and avoids the need for domain-specific ontologies.

While existing works adopt KGs and ontologies, limited research addresses KG-based reasoning, inference, and validation in the intent lifecycle. Our work addresses this gap via KGE for efficient lifecycle reasoning and CL optimization with active reporting and planning.

III. System Model

This section presents an intent model based on a multi-service orchestration framework with distinct QoS requirements. The expectations for an intent can vary from direct service delivery to service modification or monitoring [2]. In our proposed system model, we consider QoS requirements as the primary objective for a user intent. This choice is important since QoS parameters form the basis for performance evaluation of the expected and delivered service requirements for deployed intents. The QoS parameters, including priority, packet error rate, and packet latency, are selected based on the 5G QoS Indicator (5GQI) service model [42, Table V.7.4-1]. An intent formulation as a basic KG is provided, and the intuition behind utilizing KGs as a contextual information storage solution is discussed. The available information in the network management and service orchestration domains is modeled as KGs, and KGE training is done to enable inference, validation, and prediction for intent completion in the intent processing and translation pipeline. Intent completion addresses gaps in service parameters, such as service type identification and KPI enforcement, ensuring compliant deployments. In this work, intent completion ensures the fulfillment of missing service parameters in the constructed service intents. These parameters help identify the service type and the required key performance indicators (KPIs) to be ensured for the deployed services.

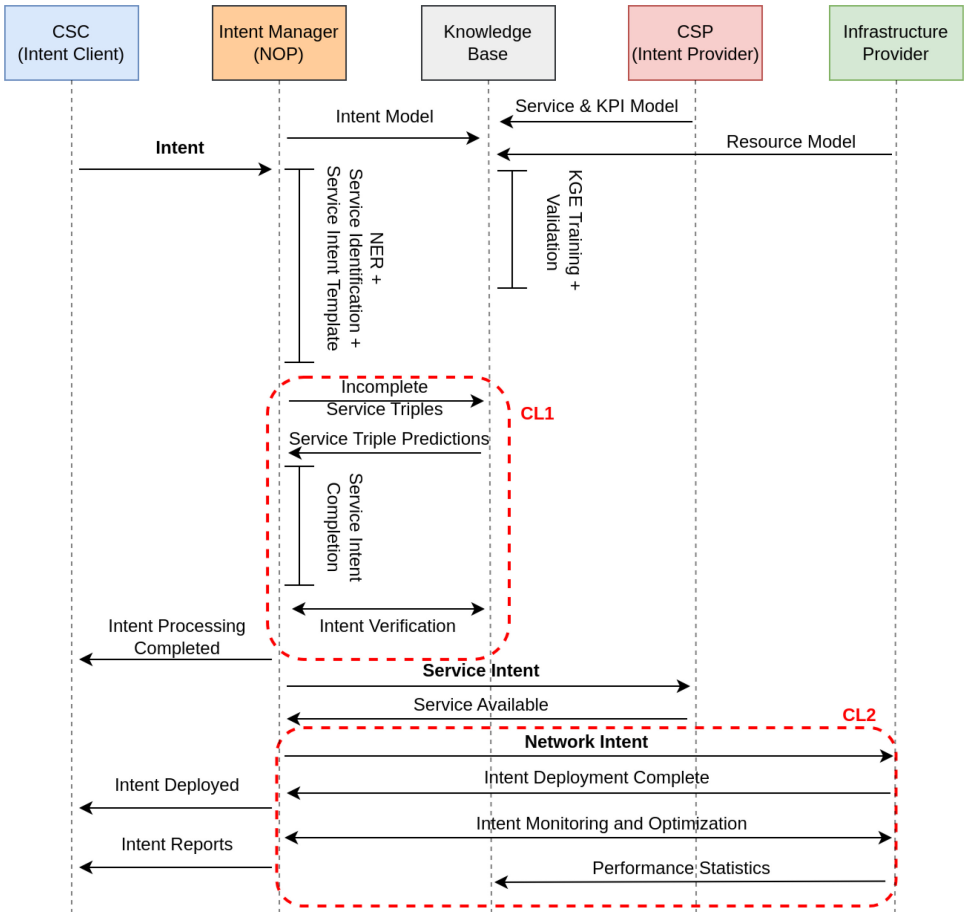


Fig. 1. Proposed IBN workflow with KGE learning for intent translation and validation.

A. KG-Driven IBN Service Orchestration Model

We propose an IBN 5G and beyond service orchestration model with relevant intent stakeholders [43]. A simple intent-based service orchestration workflow is depicted in Figure 1, where the CSC acts as the intent client and the communication service provider (CSP) acts as the intent provider. The intent manager (implemented in the network operator (NOP) domain) is responsible for the lifecycle of the intents and the communications between different intent stakeholders [1]. We consider the provisioning of a new service instance to be the objective of intent in the scope of this work. However, there can be different types of intents with the scope of monitoring or optimizing existing deployed services in the network. The services must be assured to the users upon request, irrespective of the network dynamics at any specific time. Two distinct CLs are defined: **CL1** for the processing and translation of intents using KG and **CL2** for the intent deployment and performance monitoring. This workflow outlines the critical part KG and KGE play in the processing, translation, and validation of intents before the deployment CL. The individual functionality in each phase of the intent processing workflow is discussed in Section IV.

The workflow dictates the orchestration of a service using an IBN framework proposed in Section IV. The main tasks in the workflow (depicted in Figure 1) are described as follows:

- *CSC Intent:* CSC initiates the intent for the Intent Manager hosted in the NOP. The intent may contain keywords for the service type and the relevant endpoints. The objective of the intent is to achieve the deployment of requested services as per the agreed service-level agreements (SLAs) with the NOP.
- *Named Entity Recognition (NER) and Service Template:* The Intent manager performs the NER and generation of the service intent template depicted in Figure 1. This workflow task aims to identify the service-related keywords to aid the completion of the service intent template using KGE learning.
- *KGE Training and Validation:* Used to train the Gaussian embedding model for KGE using the service orchestration knowledge graph [6]. The trained model is validated using the scoring function, and details are provided in Section IV.
- *Service Intent Completion:* This task deals with completing the service intent using the predictions obtained from link prediction of incomplete KG triples. The highest-scored predictions for the suitable service are filtered based on the context terms from the NER and verified with the service orchestration KG.
- *Network Intent Deployment:* The network intent contains all the required information regarding the service parameters and expected QoS parameters. The intent deployment

TABLE I
TABLE OF NOTATIONS

I	Set of n intents
S_i	Set of l services mapped to i_{th} intent
P_j	Set of m parameters for j_{th} service
T_j	Set of m targets for j_{th} service
$S_{i,j}$	$j_{th} \in [1...l]$ service for i_{th} intent
$P_{j,k}$	$k_{th} \in [1...m]$ parameter for j_{th} service
$T_{j,k}$	$k_{th} \in [1...m]$ target for j_{th} service
$I_i(S_{i,j}, P_{j,k})$	i_{th} intent $\in I$
$L_i(P_{j,k}, T_{j,k} \mid j = 1...l)$	Loss function for i_{th} intent
$L_i^c(P_{j,k}, T_{j,k} \mid i = 1...n)$	Cumulative loss function for all intents
n	Number of deployed intents
s_i	Scaling factor for i_{th} intent
$\mathcal{G}$	A knowledge graph component
E, R, F	A set of entities, relations, & fact triples in a knowledge graph
(h, r, t)	Fact triple, h, r, t denote head, relation, and tail
$\boldsymbol{\theta}^+, \boldsymbol{\theta}^-$	Input parameter set for +ive & -ive SPSA perturbation
$\hat{g}^t_{f_n}$	Gradient estimate for n_{th} input parameter in t_{th} iteration of SPSA
$\boldsymbol{w}^t$	Weights in t_{th} iteration of MGDA
$\boldsymbol{P_{j,k}}^+, \boldsymbol{P_{j,k}}^-$	Observed measurements for +ive and -ive SPSA perturbation
L^+, L^-	Loss calculation for +ive and -ive SPSA perturbation
ϕ	Entity and relation embeddings with KG2E

orchestrates the predicted service for the corresponding intent using the available infrastructure resources. The deployment is followed up with necessary performance reporting and optimization when required.

For a service orchestration use case, a mapping mechanism is defined to develop a relationship between the user intent and the respective deployed service. Knowledge plays a crucial role in understanding and ingesting intents; therefore, a model for a KG is defined to utilize KGs to process and map intents and services. This KG model is complemented with a knowledge representation model for the information present in the KG to aid the tasks of **service triple prediction** and **intent verification**. Moreover, a loss function is designed to assess the performance of deployed intents according to the mapped services as per available resources.

B. Intents as Knowledge Graphs

KGs consist of triples that describe facts relevant to domain information. These triples are arranged systematically using well-known structures such as OWL [8] and RDF [9]. Intents also have a specific structure with information fields describing a particular characteristic of the application use case, i.e., service characteristics in this paper. Hence, we can model intent as a KG with different entities and relationships between them as depicted in the ICM by TM Forum [10]. The mathematical model for the intent, knowledge graph learning (KGL), services, and optimization utilizes various notations summarized in Table I.

1) A Simple Knowledge Graph Model: We can represent a KG as a graph $\mathcal{G}$,

$$\mathcal{G} = (E, R, F) \tag{1}$$

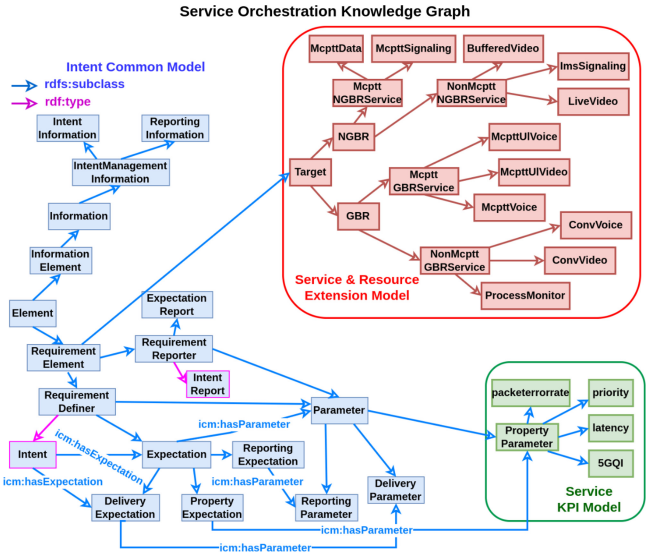


Fig. 2. Service Orchestration KG [6].

where $E = \{e_1, e_2, e_3, \ldots, e_{|E|}\}$ is the set of entities (subject or object), $R = \{r_1, r_2, r_3, \ldots, r_{|R|}\}$ is the set of relations, and $F \subseteq E \times R \times E$ is the set of fact triples and each triple is denoted as (h, r, t). Here, *h* and *t* are head and tail entities, and *r* denotes their relation. This representation allows us to model complex knowledge structures as a graph, where entities are nodes and relations are edges connecting these nodes.

An entity is an object that can be classified as a *class*, *type*, or *literal*. In a KG, *classes* (*rdfs:class*) define a hierarchical structure to organize entities into meaningful categories in a data domain. *Type* (*rdf:type*) links individual entities to their respective classes, indicating membership relation between different classes. *Literals* (*rdf:literal*) represent concrete values associated with different classes and entities, leading to comprehension and interpretation of data present in a KG[9]. In essence, *rdfs:class*, *rdf:type* and *rdf:literal* are different sets of KG vocabularies defining specific aspects of their respective entity types in a larger KG.

2) Service Orchestration Knowledge Graph: Consider the TM Forum's ICM as an example, where one of the intent triples is *(icm:Intent, icm:hasExpectation, icm:Expectation)* representing the relationship between an intent and its expectation (i.e., "*Intent has an Expectation that is Expectation*"). Here, the intent triple is complete and utilized to process an intent. In contrast, the triple *(icm:Expectation, icm:hasTarget, icm:Target)* shows the relationship (i.e., "*Expectation has a Target which is Target*") between the intent expectation and the required target to be reached. For example, during intent translation using KGs, the information regarding the expected target is updated, and afterward, the intent parameters are appended to complete the designed intent model. This is accomplished via a series of trained models using knowledge embedding [44] to predict unknown entities in intent triples.

The knowledge graph in Figure 2 represents the extended version of the ICM with augmented information from a service orchestration domain taken from our previous work [6]. These extensions include three primary information sources: services,

resources, and KPIs. The service definitions in the KG are based on the 5GQI service model [42, Table V.7.4-1] with differentiation according to the types of required resources and QoS requirements, i.e., guaranteed bit rate (GBR) or non-guaranteed bit rate (NGBR). In addition, four KPIs represent the expectations of modeled services, including latency, packet error rate, priority, and 5GQI values. We utilize this custom KG to train a KGE model to perform prediction and inference actions in the processing and translation of intents.

3) Knowledge Graph Embedding (KGE): KG analytics and reasoning benefit from embedding knowledge into a vector space. The process comprises 1) a translation of entities and relations, 2) a definition of a scoring function, and 3) a learning process for completing entity a nd relation representations. We chose a translational distance model based on Gaussian embedding (GE) [44], modeling the entities and relations as random vectors from Gaussian distributions with unique semantic meaning and variable uncertainty compared to different entities with similar semantics. For an entity or relation, the mean of its embedding denotes the center position of its semantic meanings, and the covariance matrix represents the uncertainty. The distributions for entities and relations are denoted as:

$$\begin{aligned} \mathbf{h} &\sim \mathcal{N}(\mu_h, \mathcal{C}_h) \\ \mathbf{r} &\sim \mathcal{N}(\mu_r, \mathcal{C}_r) \\ \mathbf{t} &\sim \mathcal{N}(\mu_t, \mathcal{C}_t) \end{aligned} \tag{2}$$

where $\{\mu_h, \mu_r, \mu_t\} \in \mathbb{R}^d$ and $\{\mathcal{C}_h, \mathcal{C}_r, \mathcal{C}_t\} \in \mathbb{R}^{d\times d}$ are mean vectors and covariance matrices, and $\mathcal{N}(\mu, \mathcal{C})$ denotes a multivariate Gaussian distribution. This probabilistic representation allows the model to capture the position of entities and relations in the semantic space and their uncertainty or flexibility of interpretation.

The scoring function $f_r(h,t)$ is defined to measure the correctness of a fact (h,r,t) in the embedding space. The scoring methods in knowledge graph with Gaussian embedding (KG2E) [44] are modeled as asymmetric Kullback-Leibler (KL)-divergence and symmetric expected likelihood, with the scoring function being categorized as translational distance-based. The scoring function is defined using the probability inner product as [44]:

$$\begin{aligned} f_r(h,t) &= \int \mathcal{N}_x(\mu_t - \mu_h, \mathcal{C}_t + \mathcal{C}_h) \times \mathcal{N}_x(\mu_r, \mathcal{C}_r) d\mathbf{x} \\ &\propto -\mu^T \mathcal{C}^{-1} \boldsymbol{\mu} - \ln(\det(\mathcal{C})) \end{aligned} \tag{3}$$

where $\boldsymbol{\mu} = \mu_h - \mu_r - \mu_t$ and $\mathcal{C} = \mathcal{C}_h + \mathcal{C}_r + \mathcal{C}_t$. $\boldsymbol{\mu}$ signifies the difference between mean vectors of *head*, *relation* and *tail* entities and represents the embedding of relation *r* in the context of *h* and t. In addition, $\mathcal{C}$ is the combined covariance matrix of *(h,r,t)* and captures the uncertainty and variance for the *head*, *relation*, and *tail* entities. Here, $\mathcal{N}_x$ denotes the probability density function of the Gaussian distribution.

The scoring function quantifies the plausibility of the KG triple *(h,r,t)* by analyzing the joint distribution of the embeddings for *h*, *r*, and t in a probabilistic framework. The function integrates the possible values of *x* (representing latent variables) and evaluates the agreement between the observed triple and the modeled relationships in the embedding space.

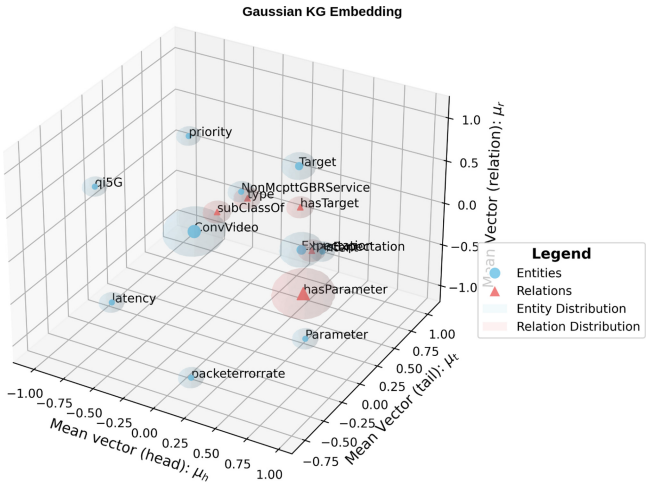


Fig. 3. Gaussian embedding sample for service orchestration KG.

4) Why Gaussian Embedding for IBN Data?: Choosing a KGE model to describe data in a higher-dimensional space requires analyzing different features of the available KG. For example, the type of entities and relationships in the KG, the complexity of the entity definitions and relationships, and the size of the dataset play an essential role in selecting a suitable KGE model.

For this paper, the custom KG consists of different service, resource, and KPI extensions to the TM Forum's ICM. The entities and relationships are highly probabilistic, and there is a very minute difference scale between different service models based on KPI literal values. This makes modeling the variance and uncertainty between the available data sensitive to changes in the Gaussian space. We have compared different available KGE models such as TransD [45], TransH [46], and compLEx [47] with the Gaussian embedding model KG2E to justify the selection of the Gaussian KGE model. The comparison results are discussed in Section V.

5) Training of KG Embedding Model: The KG utilized for training the embedding model is a modified KG model depicted in Figure 2. The training uses the KG2E [44], where each triple is evaluated to represent entities and relations in vector space accurately.

Open-world assumption (OWA) is used to prepare the dataset for the training algorithm. OWA treats the existing knowledge triples as true facts and non-observed facts as false or unknown. The input to the learning algorithm is the KG defined as the positive set of triples $F^+ = (h, r, t)$ and a negative set of triples $F^- = (h', r', t')$. Each element within the positive set, (h, r, t), constitutes a factual statement as per the KG. The negative triples augment the training process and facilitate the distinction between genuine and spurious relationships. This negative set is generated by randomly selecting head entities (h') or tail entities (t') from the positive set $(\mathcal{F}^+)$ and replacing them with alternative entities. These newly formed triples (h', r', t') are unlikely to represent true relationships within the knowledge domain. For given positive and negative triples, the entity and relation embeddings ϕ are

Algorithm 1 KG2E Training With OWA Based on [44]

Require: Observed triplets $F^+ = (h, r, t)$
Ensure: Entity and relation embeddings
while $F^+ \neq \emptyset$ **do**
  $\mathbb{S} \Leftarrow$ positive samples from F^+
  $\mathbb{N}^+ \Leftarrow \emptyset, \mathbb{N}^- \Leftarrow \emptyset$
  while $s^+ = (h, r, t) \in \mathbb{S}$ **do**
    Create a negative triplet $s^- = (h', r', t')$
    $\mathbb{N}^+ \Leftarrow \mathbb{N}^+ \cup \{s^+\}$, $\mathbb{N}^- \Leftarrow \mathbb{N}^- \cup \{s^-\}$
  end while
  Evaluate embedding based on gradient calculation from Eq. (4)
  Check for any other constraints and regularization terms
end while

learned using a pairwise ranking objective function defined as follows:

$$\min_{\phi} \sum_{f^+ \in F^+} \sum_{f^- \in F^-} max\big(0, \gamma - f_r(h,t) + f'_r(h',t')\big) \quad (4)$$

where $f^+ = (h, r, t)$ is a positive triple sample, and $f^- = (h', r', t')$ is a negative triple sample. Here, γ represents the difference between positive and negative triple values. The loss function for the KGE training improves performance for translational distance models like KG2E [47]. The learning objective in Eq. (4) is achieved through a stochastic gradient descent (SGD) based optimization algorithm given in Algorithm 1.

Negative triples are created for samples from the positive triple set F^+, and the pair is used in a minibatch training model. The embeddings for entities and relations are updated based on the calculated gradient as the ranking loss function in Eq. (4) is minimized. Figure 3 depicts a sample of the calculated and mapped embedding for entities and relations from the service orchestration knowledge graph. It projects the entities (h, t) and relations (r) from the service orchestration KG towards the Gaussian domain after the training of the KGE model. The Gaussian distributions mean vectors (μ_h,μ_t) and relations (μ_r) form the distribution axis, whereas, entities (h, t) and relations (r) form the knowledge graph representation points in the Gaussian domain. Moreover, the covariance is depicted by the ellipsoids.

C. Proposed Intent Deployment Problem

The objective of IBN is to deploy active intents in compliance with the underlying network infrastructure and resources. We model the optimization problem based on measurements from the deployment infrastructure due to the lack of a closed form for the objective function. In addition, typical gradient descent methods often encounter limitations related to high computational costs, sensitivity to initial conditions, and difficulties in handling multiple conflicting objectives. Therefore, we choose SPSA to estimate gradients using perturbations in the input parameters and observing the impact on the measured outputs of the objective function. Afterward, we choose MGDA to minimize the loss function modeled as mean squared error (MSE) in Eq. (6), although other optimization methods based on gradient descent can also be used.

1) Intent Representation: The intent definition is inspired by the relationship between intent and the corresponding ideal service to be provisioned as different stakeholders process the intent. This involves the conversion of intent in various forms until sufficient knowledge has been extracted from the KG to select a particular service. Each intent $I_i(S_{i,j}, P_{j,k}) \in I$ can have l mapped services represented as $S_{i,j}$, where $j \in [1 \cdots l]$. In addition, a j_{th} service can have m associated parameters and target values represented respectively as $P_{j,k}$, and $T_{j,k}$ where $k \in [1 \cdots m]$. The performance evaluation of an i_{th} deployed intent $I_i(S_{i,j}, P_{j,k})$ is assessed by checking the compliance of the corresponding deployed service QoS parameter $P_{j,k}$ and an expected target $T_{j,k}$ value for the QoS parameter from the trained KG model. The difference in expected and observed performance for a deployed intent is measured through a loss function.

2) Loss Function: The performance of a deployed intent is measured based on the performance metrics of the corresponding deployed services. For a given service or use case, a set of parameters defining the generic loss function is designed to characterize the performance of an i_{th} intent based on its adherence to the mapped services and KPI targets, respectively. The instantaneous loss function is based on the MSE and considers the difference between the reported and expected target values. This is shown as follows:

$$L_i\big(P_{j,k}, T_{j,k} | j = 1 \cdots l\big) = \frac{1}{l} \sum_{j=1}^{l} \big(P_{j,k} - T_{j,k}\big)^2 \quad (5)$$

where l is the total number of loss function calculations. The loss function translates the difference between the observed KPI parameter $P_{j,k}$ and the expected KPI target $T_{j,k}$ for an i_{th} intent during the intent deployment optimization phase. The system's cumulative loss for all intents is defined using the instantaneous loss function in Eq. (5). The cumulative loss is represented as follows:

$$L_i^c\big(P_{j,k}, T_{j,k} | i = 1 \cdots n\big) = \sum_{i=1}^{n} s_i \times L_i\big(P_{j,k}, T_{j,k} | j = 1 \cdots l\big) \quad (6)$$

where $s_i \in [0, 1]$ is the associated optimization scaling factor for the i_{th} intent in the system. Scaling factors are allocated based on the latency criticality of the intents to aid in prioritization between intents. For example, a strict latency (SL) intent has a higher associated scaling factor to converge quickly than a coexisting non-strict latency (NSL) intent in the system.

D. Formulation of the Optimization Problem

We formulate the intent deployment problem to minimize the difference between observed intent output parameters $P_{j,k}$ and the intent target values $T_{j,k}$. The optimization problem consists of deployed intents with a unique minimization objective regarding their respective observed and target values. The optimization problem is defined as:

$$\begin{aligned} \min \quad & L_i^c\big(P_{j,k}, T_{j,k} | i = 1 \cdots n\big) \\ & \ni \forall i \in I; \\ & P_{j,k} \geq 0; \end{aligned}$$

$$T_{j,k} \geq 0;$$
$$\sum_{i=1}^{n} s_i = 1. \tag{7}$$

IV. Proposed Closed-Loop Intent Deployment Solution

The proposed IBN framework consists of processes responsible for converting user intent to the most suitable service in the underlying network infrastructure, as depicted in Figure 4. The proposed solution is divided into a knowledge representation-based intent translation CL (**CL1**) and an intent deployment optimization CL (**CL2**) for the expression and deployment of intents.

A. Proposed IBN Framework

Firstly, we focus on the organization of knowledge in the network domain. The training of KG embedding is performed with KG2E [44] using a custom dataset of triples extracted from the KGs taken from our previous work [6] as depicted in Figure 2. The trained model is available for inference tasks such as link prediction and triplet classification. Performance statistics of deployed intents are also synchronized within the knowledge base during the validity period of the intents.

The proposed framework defines the intent processing pipeline to comprehend, represent, and convert required information from the user-defined generic intent language to specific service options offered by the service providers in the network infrastructure. A generic service template is defined as per the TM Forum's ICM [10] with the required functional components. However, information regarding service requirements and associated network parameters is incomplete and missing in the service intent template. Using the trained KGE, we extract the missing information for the entity and relation embeddings from the trained model. This information is translated into a fitness score using the scoring function defined in Eq. (4).

B. Intent Expression and Named Entity Recognition (NER)

User intents are constrained to 5G service orchestration use cases, assuming basic user knowledge of services and domain terms. We believe that the intent user will have some information regarding the available service and can use some well-known terms within a particular use case to express the intent for the network. Sample intent expressions and states are shown on the right of Figure 4. We utilize a hybrid bootstrapping learning model for NER that uses pre-defined service orchestration context terms (for example, 'service', 'video', 'voice', 'reliable', 'low-latency', etc.) for training an entity recognition model using a hidden markov model (HMM). The learning model can create variations of the service orchestration context terms and learn new entity terms from the user-defined intents over time.

Consider the intent expression in Figure 4, 'reliable voice', and 'voice consumers' are two entities recognized by our trained NER model. Here, the term 'reliable voice' provides the required type of service to be provisioned for the 'voice consumers'. This information is utilized when filtering results based on the link prediction and triple verification tasks using the trained embedding model for intent completion in Steps D, E, and F, respectively.

C. Intent Processing Pipeline

The intent processing pipeline is illustrated in Figure 4 and includes both the initial creation and the ongoing refinement of the intent model. This pipeline is integral to the intent translation and mapping model, which progresses through several well-defined steps:

1) *Step A - Processing of User Intent:* This initial step involves defining and extracting named entities from user input, which sets the search parameters and scope for the subsequent translation process.
2) *Step B - Creation of Service Intent Template:* This step develops a structured intent template based on a hierarchical model that utilizes the intent ontology outlined by TM Forum [29]. Initially, this template lacks detailed information regarding available resources, specific service mappings, and KPIs.
3) *Step C - Identification of Incomplete Intent Triples:* Here, the template's missing triples are identified and filled using service-specific data.
4) *Step D - Link Predictions from Pre-trained KGE Model:* The pre-trained KGE model predicts essential entities such as resources, parameters, and KPI values. The link prediction task provides a set of p predictions with the highest scores. The contextual information from the NER and the list of predictions are used to select an appropriate match to populate the missing data in the incomplete intent triples.
5) *Step E - Completion of Service Intent Template:* This involves integrating the predicted elements from the KGE model into the service intent template, resulting in an unverified but complete version of the network intent.
6) *Step F - Verification of Network Intent:* The final step validates the completeness and accuracy of the network intent by classifying and scoring the modified intent triples using the predefined scoring function specified in Eq. (3). This is a typical triple classification problem within the KGE modeling framework. The classification uses the pre-trained KG2E model to label the completed service intents as having valid or invalid triples. The invalid service intents are discarded, and **CL1** ensures the selection of a different option until a verified service intent is constructed.
7) *Step G - Deployment and Optimization:* Upon completion of Step F, the network intent contains all the necessary information for deployment of the requested mapped service. The deployment is performed by extracting the relevant information from the Network Intent in the underlying 5G NSA infrastructure. In the meantime, the monitoring of the deployed intents is also performed using **CL2**, and the compliance status is updated for the existing intents in the knowledge base in the form of structured intent reports. These

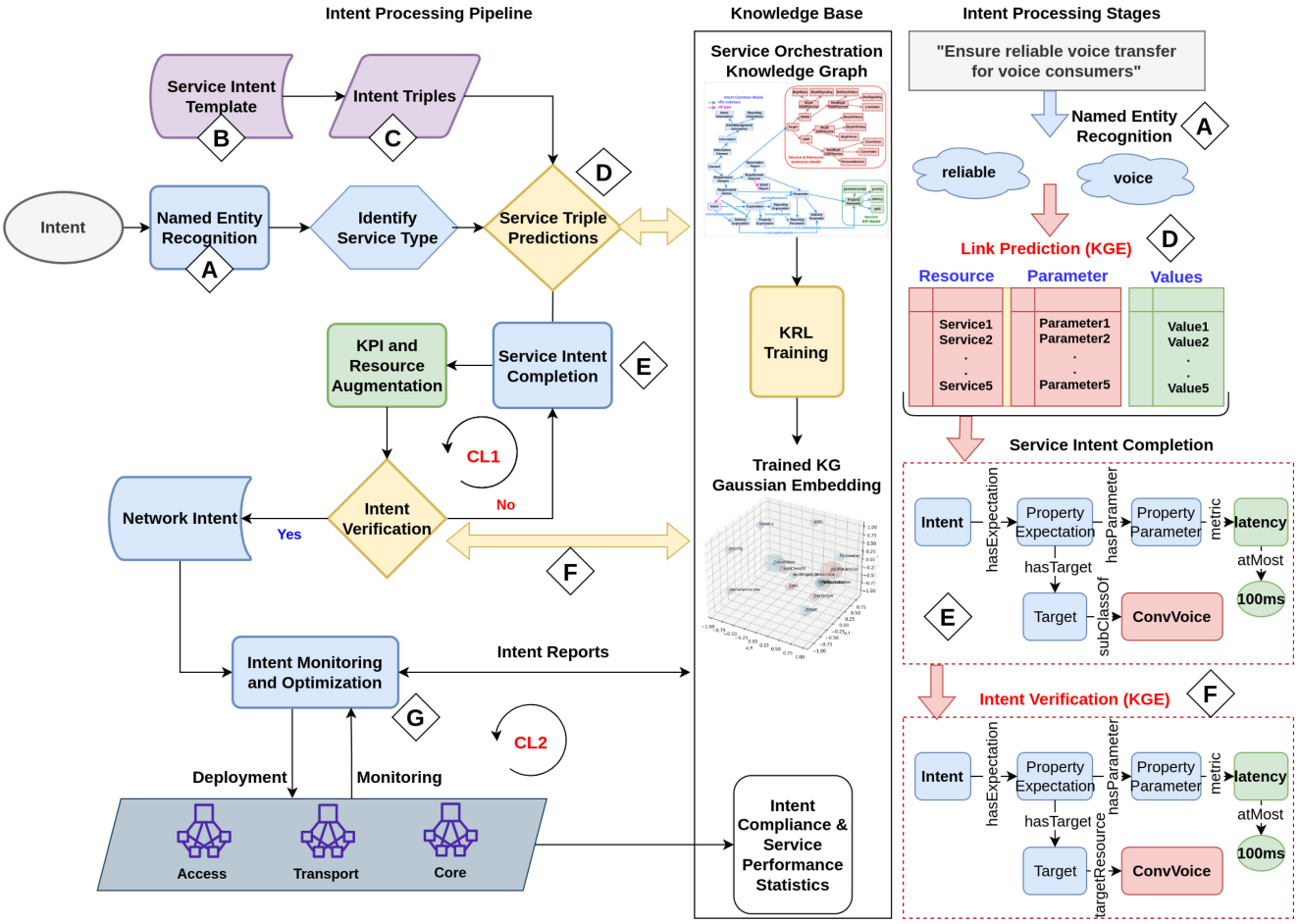


Fig. 4. Proposed intent processing pipeline with KG-based learning.

periodic intent reports are generated to track the status of the deployed intents over time and reconcile the performance against agreed SLAs. This approach ensures systematic updating and verification of the intent model, aligning it closely with operational requirements and specifications.

D. Solution to the Intent Deployment Problem

The solution to the optimization problem Eq. (7) is possible through the integration of SPSA and MGDA. This optimization approach builds on the efficiency of SPSA in handling high-dimensional spaces and the adeptness of MGDA at managing multiple objectives. An IBN system must address the simultaneous deployment of multiple intents with differing objectives. MGDA allows the computation and evaluation of gradients for each objective simultaneously and ensures solutions satisfying multiple objectives. A brief description of SPSA and MGDA is provided as follows, building up to the design of a combined optimization algorithm.

1) Simultaneous Perturbation Stochastic Approximation (SPSA): SPSA [12] is designed for optimizing complex systems with multiple unknown parameters. SPSA enables improved efficiency in high-dimensional optimization problems. The algorithm requires only two measurements of the objective function in each iteration to estimate the relevant gradient irrespective of the system dimensionality. Moreover, stochastic perturbations in the input parameters avoid falling into local optima during the iterations.

SPSA aims to determine an optimal input vector θ^* that minimizes a multivariate function $f\ (\theta) : \mathbb{R}^n \rightarrow \mathbb{R}$, defined as [12]:

$$\theta^* = argmin_\theta f(\theta) \tag{8}$$

where $\theta \in \mathbb{R}^n = (\theta_n)_{0 \leq n \leq N-1} = \{\theta_0, \theta_1, \ldots, \theta_n\}$ is the input parameter set with dimensionality of N. $f(\theta)$ represents the objective function that needs to be minimized. We consider a MSE objective function for SPSA. In each iteration t, an estimate for the gradient $\hat{g}_{f_n}^t$ is calculated for a given stochastic gradient approximate set $\hat{g}_f = (\hat{g}_{f_n}^t)_{0 \leq n \leq N-1}$. A single gradient estimate for n-th parameter is obtained using [12]:

$$\hat{g}_{f_n}^t = \frac{f\left(\theta_n^t + c^t \Delta_n^t\right) - f\left(\theta_n^t - c^t \Delta_n^t\right)}{2c^t \Delta_n^t}. \tag{9}$$

Here, $c^t = c/t^\gamma$ is the size of the perturbations, whereas, Δ_n^t is the random factor perturbation calculation for n-th parameter in t-th iteration and $\Delta^t \in \mathbb{R}^n\ \mathcal{U}(-1, 1)$. This gradient estimation is a key feature of SPSA, allowing it to approximate gradients using only two function evaluations, regardless of the number of parameters.

Using Eq. (9), the perturbation is applied to the parameter set $\boldsymbol{\theta}$ in each iteration as follows:

$$\boldsymbol{\theta}^{t+1} \leftarrow \boldsymbol{\theta}^{t} - \Delta^{t}\hat{\boldsymbol{g}}^{t}_{f_n} \tag{10}$$

where $\Delta^t = \Delta/(t+A)^{\mu}$ is the step size for the perturbations in each iteration t. This update rule adjusts the parameters toward the steepest descent, as estimated by the SPSA gradient approximation. *Spall* [12] suggests the values for γ and μ to be 0.101 and 0.602, respectively, for optimal performance of SPSA. However, we observe that varying the values adaptively between the practical and asymptotic values, as suggested in [12], improves convergence for different sets of deployed intents in the system. In addition, a modified version of SPSA that uses Hessian for second-order estimation of the gradient is more accurate than a simple SPSA and less prone to noise in the objective function measurements [48]. The gradient estimation is also weighted against the scaling factors when calculating the new value of $\boldsymbol{\theta}$ at the end of an optimization iteration.

2) Multiple Gradient Descent Algorithm (MGDA): Simultaneous optimization of multiple goals is challenging with numerous possible solution spaces. In MGDA [13], Pareto efficiency is considered foremost since it enables the collective optimization of all objectives. A solution to a problem is Pareto optimal when no other objective can be improved without worsening another objective. The gradients for all objective functions are calculated at a solution point, and a search for a common descent direction is performed. This is achieved by exploring the Pareto front (set of all Pareto optimal solutions) by adjusting the objectives' weights. The iterative updates in the MGDA algorithm allow establishing a Pareto set consisting of all Pareto optimal points in the solution space. The aggregated gradient information from different objectives enables the discovery of optimal or near-optimal solutions for all objectives simultaneously.

The MGDA allows the calculation of weighted gradients for all the active intent objectives. Afterward, the weight that minimizes the loss function is chosen as the optimal weight in each iteration t defined as [13]:

$$\boldsymbol{w}^{t} = \min_{w_1,w_2,\ldots,w_N} \left\| \sum_{n=1}^{N} w_n \hat{g}^{t}_{n} \right\|^2_2$$
$$\ni \sum_{n=1}^{N} w_n = 1, w_n \geq 0 \quad \forall n. \tag{11}$$

Here, $\hat{g}_n$ and w_n represent the gradient and associated weights for the n-th intent. This optimization problem finds the set of weights that minimizes the squared norm of the weighted sum of gradients, ensuring a common descent direction for all objectives. We employ a modified version of the MGDA that treats the objectives according to the scaling factors of optimization targets $(T_{j,k})$ for the respective intents. This modified MGDA does guarantee an optimal solution for all intents and prioritizes an optimal deployment for the SL intents with lower $(T_{j,k})$ values. Using Eq. (11), the input parameter set $\boldsymbol{\theta}^{t+1}$ for the objective measurement function is updated now as:

$$\boldsymbol{\theta}^{t+1} \leftarrow \boldsymbol{\theta}^{t} - \Delta^{t}\boldsymbol{w}^{t}\hat{\boldsymbol{g}}^{\boldsymbol{t}}_{\boldsymbol{f}}. \tag{12}$$

This update rule combines the SPSA gradient estimation with the MGDA-optimized weights to adjust parameters in a direction that improves all objectives simultaneously. The input parameters $\boldsymbol{\theta}^t$ are changed slightly according to the perturbation set, and the difference between randomly perturbed output parameters is used to calculate the gradients for the active intents. Once the gradients are available, MGDA takes over and performs the multi-objective optimization for the active intents.

Algorithm 2 Intent Optimization Algorithm Based on SPSA and MGDA

Require: SPSA parameters θ_0, θ^{max}, θ^{min}, Δ, A, μ, c, γ, $\boldsymbol{T_{j,k}}$
Ensure: θ^*
1: **for** $t = 0, 1, \ldots, \tau$ **do**
2: Calculate the SPSA step size Δ^t:
$$\Delta^t \leftarrow \frac{\Delta}{(t+A)^{\mu}}$$
3: Calculate the SPSA perturbation size c^t:
$$c^t \leftarrow \frac{c}{t^{\gamma}}$$
4: Compute the perturbed input parameter sets:
$$\boldsymbol{\theta^+} \leftarrow \theta^t + c^t\Delta_t$$
$$\boldsymbol{\theta^-} \leftarrow \theta^t - c^t\Delta_t$$
$$\boldsymbol{h^+} \leftarrow \theta^t + 2 \times c^t\Delta_t$$
$$\boldsymbol{h^-} \leftarrow \theta^t - 2 \times c^t\Delta_t$$
5: Obtain measurements for the perturbed sets:
$$\boldsymbol{P_{j,k}}^{+} \leftarrow \mathcal{O}(\boldsymbol{\theta^+})$$
$$\boldsymbol{P_{j,k}}^{-} \leftarrow \mathcal{O}(\boldsymbol{\theta^-})$$
6: Estimate the Hessian for the perturbed sets:
$$\hat{H} = \frac{\mathcal{O}(\boldsymbol{h^+}) - 2 \times \boldsymbol{P_{j,k}}^{+} - \mathcal{O}(\boldsymbol{h^-})}{(c_t^2\Delta_t^2)}$$
7: Compute the losses with Eq. (5) for measurements and target values $\boldsymbol{T_{j,k}}$ for active intents I:
$$\boldsymbol{L}^{+} = MSE(\boldsymbol{P_{j,k}}^{+}, \boldsymbol{T_{j,k}})$$
$$\boldsymbol{L}^{-} = MSE(\boldsymbol{P_{j,k}}^{-}, \boldsymbol{T_{j,k}})$$
8: Estimate the gradient $\hat{\boldsymbol{g}}^{\boldsymbol{t}}_{\boldsymbol{f}}$ using measurements:
$$\hat{\boldsymbol{g}}^{\boldsymbol{t}}_{\boldsymbol{f}} \leftarrow \frac{(\boldsymbol{L}^{+} - \boldsymbol{L}^{-})}{2c^t\Delta^t} \times (\sqrt{|\hat{H}|})^{-1}$$
9: Using MGDA, find weights using Eq. (11): $\boldsymbol{w}^t$
10: Update the parameters using the estimated gradient:
$$\boldsymbol{\theta}^{t+1} \leftarrow \boldsymbol{\theta}^{t} - \Delta^{t}\boldsymbol{w}^{t}\hat{\boldsymbol{g}}^{\boldsymbol{t}}_{\boldsymbol{f}}$$
11: Early stopping and convergence conditions:
12: **if** $\theta^{t+1} - \theta^t \leq \epsilon$ **then**
13: *break;*
14: **else**
15: *continue;*
16: **end if**
17: **end for**
return $\theta^* = \boldsymbol{\theta}^{t+1}$

3) Proposed Intent Optimization Algorithm: The proposed algorithm based on SPSA and MGDA for planning and deployment of intents is listed as Algorithm 2. The algorithm starts with SPSA and the calculation of the perturbed input parameter vector $\boldsymbol{\theta} = [\boldsymbol{\theta^+}, \boldsymbol{\theta^-}]$ (*line# 3-5*). Afterwards, objective function measurements $P_{j,k} = [P^{+}_{j,k}, P^{-}_{j,k}]$ are obtained via a simulated 5G NSA network (*line# 6*). The Hessian is calculated based on the observed objective function measurements and Hessian perturbed sets (h^+, h^-) (*line#7*). Loss function values are obtained for both sets of observed

measurements from the objective function (*line# 8*). The gradient is estimated (*line# 9*) using the perturbed sets, Hessian, and the calculated losses. Using SGD, the weights that determine the minimum descent direction are determined (*line# 10*). At this point, the input parameter set is updated using the perturbations from the SPSA and the direction of loss function minimization from the MGDA (*line# 11*). The criterion for the convergence and early stopping is evaluated with a decision on the continuation or termination of the algorithm (*line# 12-15*).

4) Convergence Criterion: The convergence of this combined algorithm depends on several factors:

- *Perturbation Size:* The perturbation sizes in SPSA impact the bias and variance of the gradient and Hessian estimates. Properly chosen, they diminish across iterations to balance exploration and exploitation, affecting convergence speed and stability.
- *Learning Rates:* Adaptation of learning rates, particularly in stochastic and noisy estimations, is crucial. These should ideally decrease as per specific schedules to ensure convergence.
- *Mutual Descent Direction:* In MGDA, identifying a common descent direction that adequately balances improvements in all objectives is challenging and critical for convergence. The algorithm must navigate the trade-offs between different objectives effectively.

We have included stopping conditions considering the changes in loss function calculation and estimated gradients evaluated in each optimization algorithm iteration. The convergence and early stopping criterion are implemented for the combined SPSA and MGDA algorithm. The convergence is ensured using the defined intent thresholds, continuous minimization in calculated losses, and tolerance levels for the estimated gradients for the deployed intents [48]. The algorithm is stopped early if there is no change in the calculated loss for 10 iterations, and the gradient tolerance level is defined as $10e^{-6}$. The gradient tolerance level ignores changes in the estimated gradient below this level and stops the algorithm if this happens continuously for several iterations. Moreover, the scaling factor for MGDA is adjusted according to the compliance between the parameter $P_{j,k}$ and target $T_{j,k}$ values for each intent. For example, the scaling factor is decreased once an intent's measured parameter $P_{j,k}$ becomes less than or equal to the preset target value $T_{j,k}$. This skews the optimization in favor of the remaining non-compliant intents in the system, focusing on their optimization and eventually leading to intent compliance.

5) Complexity Analysis: The complexity of the proposed SPSA and MGDA algorithm is dependent on multiple factors:

- *Gradient estimation (SPSA):* Each step of the perturbation calculation has a dimensionality of *n*, equal to the number of intents. The perturbation steps are calculated twice (Line#4), resulting in a $\mathcal{O}(t(2n))$ cost per iteration of the algorithm for a total of *t* iterations. However, since the computational cost for generating and processing these perturbations is linear in *n* and the total number of iterations *t* is bounded by the early stopping criterion, the effective cost of this step is $\mathcal{O}(n)$.
- *Perturbed measurements:* Each evaluation of the perturbed vector requires querying the ns3 simulator for observed performance metrics (Line#5). The time complexity of this step is $d_{sim}(n)$ for *n* intent objectives. The total computational cost for this step is represented as $\mathcal{O}(d_{sim}(n))$, and it may scale linearly or non-linearly based on the size of the network, traffic model, and available resources.
- *Objective optimization (MGDA):* After collecting all the perturbed measurements, MGDA is used to perform multi-objective optimization (Line#9). This involves solving a quadratically constrained quadratic program (QCQP) or a convex optimization problem. The worst-case complexity of solving such a convex problem for *n* objectives is $\mathcal{O}(n^3)$, which is polynomial scaling.

The overall computational complexity comes out to be $\mathcal{O}(n) + \mathcal{O}(n^3) + \mathcal{O}(d_{sim}(n)) = \mathcal{O}(n^3)$ for *t* iterations and *n* objectives of the algorithm. Hence, the computational complexity has a polynomial increase in cost when *n* scales.

V. Evaluating the Intent Processing With KG Embedding and Model Training

This section describes the evaluation setup and results for intent processing and conversion to a set of target services KPIs by utilizing a trained KG embedding model. The contextual information available from the underlying network and the service providers in the form of service requirements and KPIs plays a vital role in the KGL training and subsequent predictions of service mapping for the intents.

A. Knowledge Graph Training

Knowledge graphs represent the available information in the network consisting of the service model, KPIs, and resource type. We utilize the knowledge graph extension model depicted in Figure 2 to train the KG2E Gaussian representation for triples. Table II offers a comprehensive overview of the performance metrics measured during training and validation of the KG2E model. We have trained different KGE models (*transD* [45], *transH* [46], *compLEx* [47]) for performance comparison using the custom KG in Figure 2. Here, *transD* and *transH* represent the entities and relations in vector spaces, whereas *complEx* utilizes a complex vector space. Evaluations are conducted using rank-based and classification-based methods to thoroughly analyze the performance of the IKG across service triple prediction and intent verification scenarios. KG2E seems to outperform them all, and we select it due to its dependency on the variance and uncertainty modeling in the Gaussian embedding space. This analysis provides both service triple prediction metrics and triple classification metrics, useful in **Steps D** and **E** of the intent processing pipeline depicted in Figure 4.

1) Performance of Service Triple Prediction Task (Step D): Service triple predictions are facilitated by applying KGE link prediction techniques, with performance evaluation via a rank-based evaluator. This encompasses two distinct link prediction tasks: 1) **right-side prediction**, where the tail entity is predicted given a pair consisting of the head entity and

TABLE II
TRAINED KGE MODEL PERFORMANCE

Link Prediction Metrics					Classification Metrics				
Metric\Model	KG2E [44]	transD [45]	transH [46]	compLEx [47]	Metric\Model	KG2E [44]	transD [45]	transH [46]	compLEx [47]
# of triples	1575	1575	1575	1575	accuracy	81.60	69.11	50.97	54.17
MR	4.5	9.8	5.6	15.46	f1_score	0.9830	0.9765	0.9765	0.9764
hits@1	0.5882	0.3653	0.4615	0.2884	TPR	0.6718	0.6048	0.4409	0.5043
hits@3	0.7647	0.5192	0.75	0.4615	TNR	0.9809	0.9765	0.9658	0.9713
hits@5	0.8235	0.6346	0.7884	0.5192	FPR	0.3282	0.3952	0.5591	0.4957
hits@10	0.9118	0.7115	0.8461	0.6153	FNR	0.0190	0.0235	0.0342	0.0287

relation, denoted as *(h, r, ?)*, and 2) **left-side prediction**, where the head entity is predicted given a pair consisting of the tail entity and relation, denoted as *(?, r, t)*. For both tasks, entity or relation scores are computed using the designated scoring function presented in Eq. (2). After scoring, entities or relations are ordered in descending score value to ascertain the rank of the correct choice within the KG. The primary metrics employed for the rank-based evaluation include: 1) **Mean Rank (MR)**, which represents the average position of the correct entities and relations within the ranked list, and 2) *Hits@p*, defined as the frequency at which valid entities or relations appear within the top **p** positions of the ranked list. Each service triple prediction task produces a list of top **p** candidates for triple completion of the service intent. The best possible predictions are selected based on the achieved score and the available context from the NER. An optimal embedding model is characterized by a minimal Mean Rank and a maximal Hits@p metric for various thresholds of **p**.

The results show a total of *1575* triples were evaluated during the training, testing, and validation phases. The model achieved a *Mean Rank* of *4.5*, which indicates the average rank of the true triple among all triples ranked by the model's scoring function. A lower mean rank suggests a better performance, as the correct triples are closer to the top of the list. Regarding the *Hits@K* metrics, the model performed increasingly better as the number of top K predictions increased: *Hits@1* signified that about 58.82% of the true positives are ranked first by the model. When considering the first three places denoted by *Hits@3*, true positives increased to 76.47%. Furthermore, *Hits@5* and *Hits@10* showed further improvement, with a success rate of 82.35% and 91.18%, respectively. These link prediction metrics highlight the model's capacity to rank true triples highly, but also point out that the precision drops notably when only the top result is considered. This suggests an additional need for further model refinement to improve accuracy at higher precision levels. We handle this through available NER context terms when choosing the predictions received in **Step D**.

2) Performance of Intent Verification Task (Step F): We use KG2E model's triple classification task to score the intents using a classification-based evaluator. The intents are either valid and compliant with the structure of KG or invalid. However, we first define basic performance metrics from a classical machine learning (ML) classification model and their meaning for our scenario as follows:

- True positive (TP): A correctly predicted intent as valid that is also valid as per the KG.
- True negative (TN): A correctly predicted intent as invalid that is also invalid as per the KG.
- False positive (FP): An incorrectly predicted intent as valid that is invalid as per the KG.
- False negative (FN): An incorrectly predicted intent as invalid that is valid as per the KG.

The set of metrics for a typical classification task are 1) *Accuracy*: ratio of the number of correct classifications to the total number, 2) *f1-score*: harmonic mean of precision calculated as $TP/(TP+FP)$ and recall calculated as $TP/(TP+FN)$, 3) *True positive rate (TPR)*: the probability of correctly predicting a valid intent, 4) *True negative rate (TNR)*: the probability of correctly predicting an invalid intent, 5) *False positive rate (FPR)*: the probability of incorrectly predicting a valid intent, 6) *False negative rate (FNR)*: the probability of incorrectly predicting an invalid intent.

For the KG2E trained model, we focus on the best-observed performance, where *accuracy* reached 81.60%. This indicates the model's strong ability to classify most instances correctly. The *f1-score* was exceptionally high at 0.9830, which is particularly notable because it balances the precision and recall, suggesting both are high. *TPR* was 0.6718, implying that the model is somewhat effective in identifying positive valid intent instances, but there is room for improvement in recognizing true positives. *TNR*, however, was extremely high at 0.9810, indicating excellent performance in identifying negative invalid intent instances. Moreover, a *FPR* of 0.3282 suggests that the model falsely identifies negative invalid intent instances as positive valid intent instances relatively frequently. However, *FNR* was very low at 0.0190, suggesting that the model is very effective at not missing true positives of valid intents.

B. Discussion

1) Service Intent Completion (Step E): NER provides the relevant context for the user intent that can be used to determine the type of service mapping required from the available service offerings. The process of hierarchical service triple predictions is depicted in Figure 5. Completing the service intent triples will help translate the intent into a mapped service. The incomplete triples in the service intent are listed as follows:

- *Triple#1:* (*icm:PropertyExpectation*, *icm:hasTarget*, **???**)
- *Triple#2:* (*icm:ConvVoice*, *icm:targetResource*, **???**)
- *Triple#3:* (*kpi:latency*, *icm:valueBy*, **???**)

These triples consist of known *head* entity and *relation*, whereas *tail* (**???**) entities are missing and need to be populated using the pre-trained model to perform the service intent completion task. **Triple#1** expects a valid service

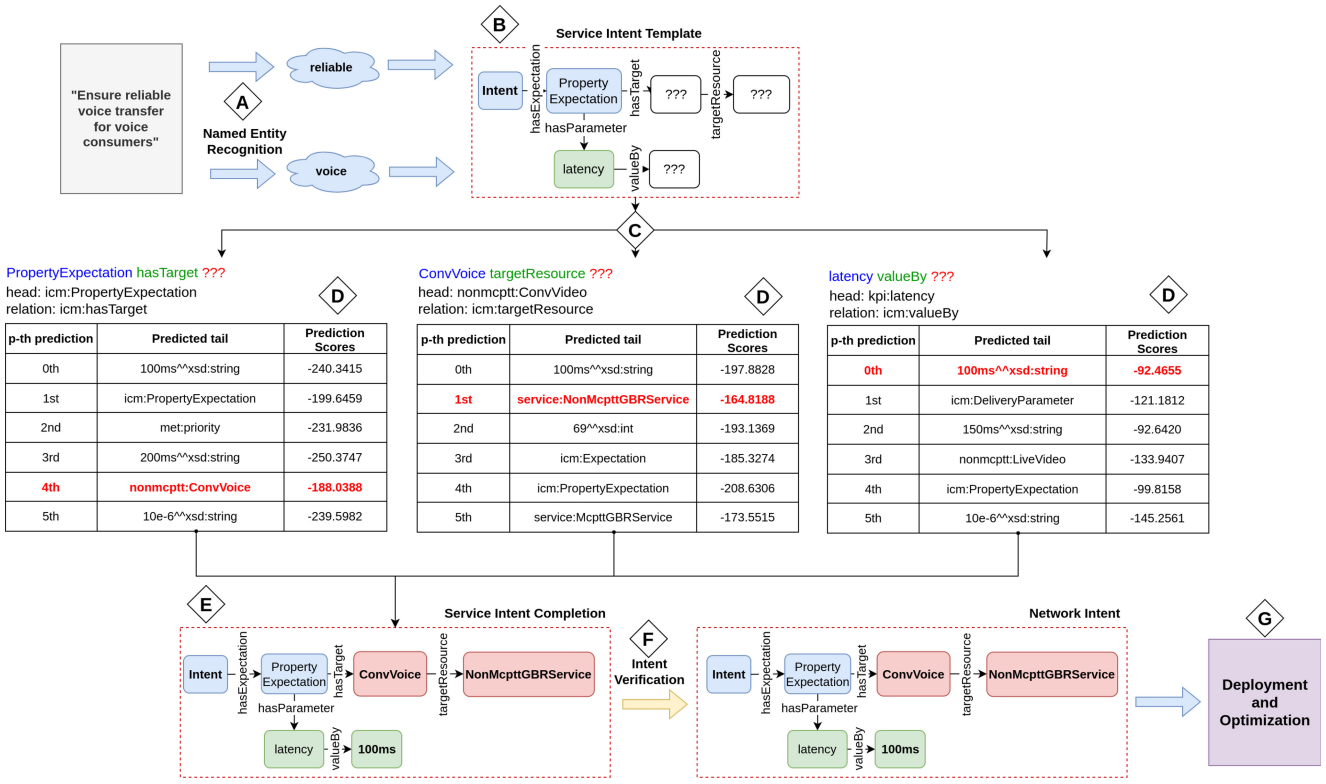


Fig. 5. Service triple prediction example with completed service intent.

model from the KG and finds its relevant prediction (i.e., *ConvVoice*) from the list of predictions using the prediction score and the context from the recognized named entities in the original intent as shown in Figure 4. Similarly, **Triple#2** and **Triple#3** determine their missing tail entities (i.e., *service:NonMcpttGBRService*, and *100ms^^^xsd:string* respectively) using the highest prediction score and the available context. The predictions for the incomplete triples are listed in Figure 5. Each triple corresponds to the service, resource, and parameter determination for completing the service intent template. The ranked predictions for each component are determined according to the scoring function defined in Eq. (3). Moreover, filtering suitable predictions from the list of predictions is also enabled through intent verification (using process **F**) from the intent processing pipeline.

2) Network Intent Verification (Step F): There is always a possibility of error in the selected predictions for the service intent completion. The intent processing and interpretation pipeline is consolidated with the introduction of the intent verification step, aided by the concept of triple verification [49] with the trained KGE model. However, the probability of incorrect predictions appearing in the recommendations from the trained embedding model significantly decreases for a comprehensive knowledge graph that embodies a maximum number of concepts in a domain. This problem is addressed with the introduction of intent verification, where the triples with the highest score are considered the most valid options for completing intents. Consider the second example from Figure 5, where the incomplete triple *(nonmcptt:ConvVoice, icm:targetResource, ???)* is completed by the highest-ranked prediction *(service:NonMcpttGBRService)*. The prediction *service:McpttGBRService* can also be validated since it has the second-highest prediction score amongst the available recommendations. However, the intent verification ensures that the intent triple completion with *service:McpttGBRService* is invalid since it cannot be validated from the KG in Figure 2. In contrast, the remaining predictions do not fulfill the fundamental ontology of the KG. They are not validated, prompting the need to select a valid intent triple completion through another prediction. The completed and validated service intent is termed the network intent per TM Forum's terminology and contains the required service parameters for deployment in the underlying network infrastructure.

VI. Evaluating the Intent Deployment and Optimization

This section describes the intent deployment scenario using the available target service KPIs and a set of network control parameters through SPSA and MGDA-based optimization and intent monitoring control loops.

A. Simulation Model

The intents are deployed in a simulated 5G NSA infrastructure developed in network simulator 3 (ns-3) with user groups requiring differentiated service delivery objectives specified through respective intents as shown in Figure 6. The deployed intents have a 1-1 mapping for the deployed services. A Poisson arrival process is modeled for intents through the

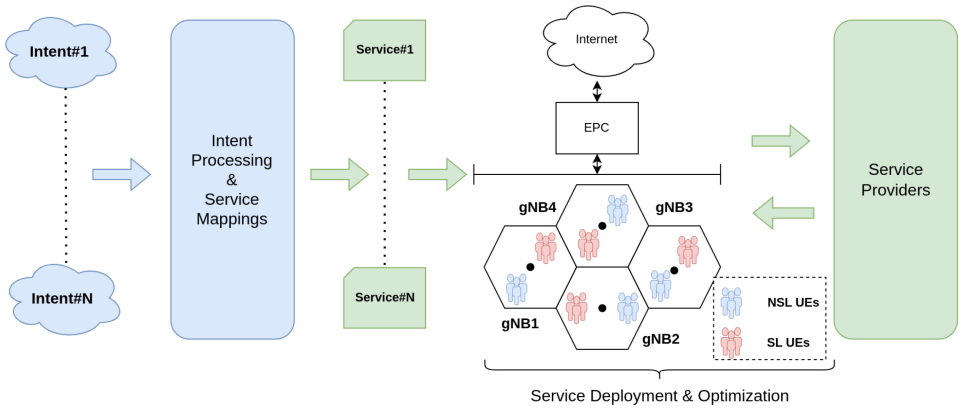


Fig. 6. Simulation Setup for the Proposed Intent Deployment Model.

intent processing pipeline, and the corresponding mapped services are provisioned with the required resources.

The proposed intent deployment and optimization algorithm requires sampling performance metrics for the orchestrated services from the underlying infrastructure. The optimization process reaches an ideal operation point for coexisting services by introducing variable traffic shaping for available service streams using active queue management (AQM). This traffic shaping is achieved by optimizing queue latency targets for various service flows in FlowQueue-Controlled Delay (FQ-CoDel) to achieve the target end-to-end (E2E) latency and respective level of QoS.

The designed simulated model aims at creating a stable and reproducible experimentation environment. It takes into account conditions and metrics selected from 5GQI service model specifications [42, Table V.7.4-1] and simulates them in ns-3. This design can be further extended to replicate realistic network management and control loops, where the deployment of services may differ.

The following steps are taken during the simulation to orchestrate a service:

1) Mapping of user intents to available service offerings using the intent processing pipeline and knowledge inference.
2) Determination of the state of resources in the simulated 5G NSA network for service orchestration and availability of service providers.
3) Deployment of services in the 5G NSA network with latency targets as QoS assurance KPIs.
4) Optimization is performed using Algorithm 2 with $\boldsymbol{\theta}$ being the traffic shaping latency targets for FQ-CoDel and observed E2E latency for services as the output parameters $\boldsymbol{P_{j,k}}$. The target E2E latency for respective services acts as the target $\boldsymbol{T_{j,k}}$ for the calculation of the loss function defined in Eq. (5).
5) Introduction of new user intents triggers the optimization process, and a converged solution using MGDA is obtained for assurance of QoS for all deployed intents (as mapped services).

B. Example Use Case: Voice and MCPTT Intents

This scenario considers *ConvVoice* and *mission-critical push-to-talk (MCPTT)* services deployed using the available information in the KG [42] at the same simulation time instance $t = 0ms$. The targets are extracted from the intent as $T_{1,1} = 100ms$ for the *ConvVoice* and $T_{2,1} = 75ms$ for the *MCPTT*. The convergence time is measured as the average time for Algorithm 2 to satisfy the early stopping or loss-based convergence criterion.

1) Convergence Performance: After the intents are deployed at $t = 0ms$, it is observed that the difference between observed and target latencies continues to minimize as the optimal gradient descent direction is determined using SPSA estimates by the MGDA part of Algorithm 2. The convergence for the SPSA and MGDA is achieved after $600ms$ (sixth iteration of the algorithm) as shown in Figure 7(a). A detailed analysis of this scenario is presented as an empirical cumulative distribution function (ECDF) in Figure 7(b). The results highlight the distribution of *average E2E latency* and *average convergence time* for the deployed intents to determine the performance of the intent compliance and optimization process. The plot shows that the algorithm converges for 95% and 97% of the independent simulation runs within 1600 ms and $2600ms$, respectively.

2) Re-Optimization and Re-Convergence Performance: In this scenario, the *ConvVoice* intent is deployed similarly to the previous scenario at $t = 0ms$, whereas the *MCPTT* intent is deployed afterward at $t = 600ms$. The deployment instance of the *MCPTT* intent serves as an event whereby the SPSA and MGDA algorithm convergence is disturbed due to the change in network dynamics. This intent arrival signifies introducing a high-priority intent requiring immediate optimization and allocation of available network resources, interrupting the converged non-critical *ConvVoice* intent. The following steps are taken to ensure the convergence of Algorithm 2:

- Processing and handling of new *MCPTT* intent;
- Modification of the scaling factors (*0.8* for *MCPTT* and *0.2* for *ConvVoice*);
- Iterative optimization of the existing intents as per their objectives;
- Monitoring the loss minimization for the deployed intents.

Figure 8(a) provides a single simulation sample result from the list of independent simulation runs. It is observed that the difference between observed E2E latency $(P_{1,1})$ and

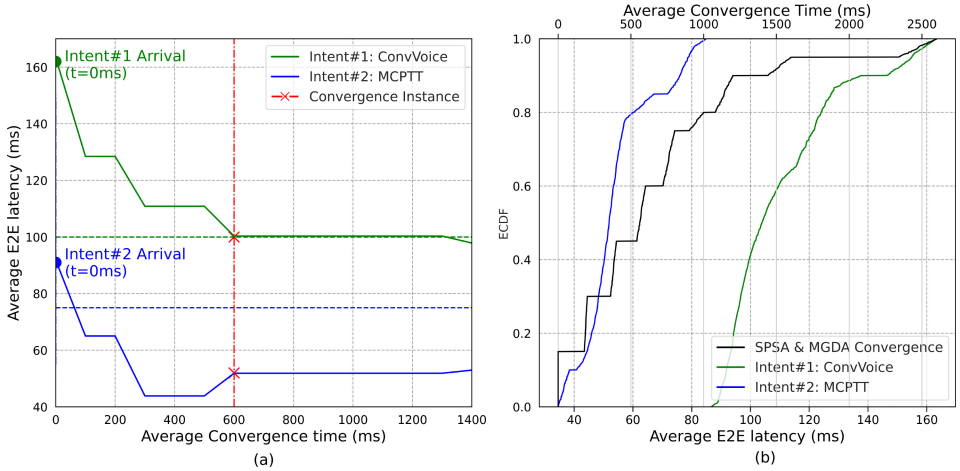


Fig. 7. Compliance of ConvVoice and MCPTT intents both deployed at $t = 0ms$.

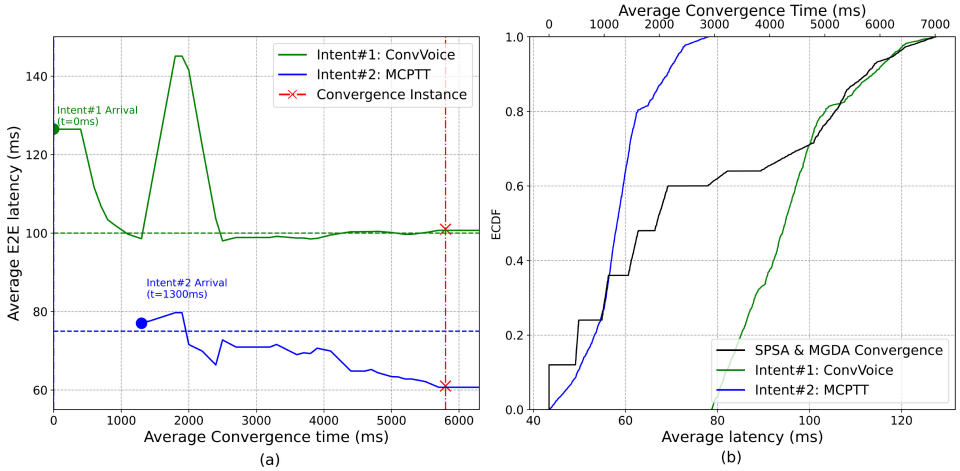


Fig. 8. Compliance of ConvVoice and MCPTT intents deployed at $t = 0ms$ and $t = 1300$ ms, respectively.

the target E2E latency $(T_{1,1})$ decreases, and the *ConvVoice* intent becomes compliant at approximately $t = 1250ms$. The introduction of *MCPTT* intent at t= 1300ms triggers the re-optimization of MGDA due to the addition of an optimization objective for the newly added intent in the system. This causes a significant increase in the observed E2E latency $P_{1,1}$ for the conversation voice intent, whereas the observed E2E latency for the *MCPTT* intent decreases. The *MCPTT* intent becomes compliant at approximately $t = 1980ms$. Still, the optimization process continues with the loss minimization for the deployed intents mainly due to a larger difference between observed and target E2E latencies of the *ConvVoice* intent. The convergence criterion is met at $t = 6000ms$ for the deployed intents, and the re-optimization is concluded.

The ECDF provided in Figure 8(b) shows average convergence time and E2E latencies for the deployed *MCPTT* and *ConvVoice* intents. The performance of the combined SPSA and MGDA algorithm shows that 60% and 80% of the simulations converge after 2000ms and 5200ms, respectively. Moreover, the plot also depicts the performance of the deployed *MCPTT* intent being fully compliant with its target E2E latency $T_{2,1} = 75ms$ for about 98% of the simulation runs. This is again attributed to the skewed optimization within MGDA due to the higher scaling factor $s2 = 0.8$ allotted to the SL service intents. We also observe that the maximum observed E2E latency $P_{2,1}$ for the *MCPTT* comes at 78ms, 4% higher than the target E2E latency $T_{2,1}$. Approximately 70% of the *ConvVoice* intents comply with $T_{1,1} = 100ms$, and a maximum bound for the observed latency is 123ms. These larger convergence times are due to the introduction of the *MCPTT* intent and redirecting resources to accommodate the MCPTT service at the expense of the conversation voice service.

3) Impact of MGDA Scaling Factor: The convergence of the proposed algorithm is evaluated by conducting a sensitivity analysis of the MGDA scaling factors (*s1, s2*) for individual *ConvVoice* and *MCPTT* intents in the system. The analysis provides several critical operation points for the optimization algorithm and the relationship between the scaling factor and optimization performance. Scaling factors help to identify the importance of an intent over other intents. We treat the criticality and lowest latency target values $\boldsymbol{T_{j,k}}$ as the highest priority, and a higher scaling is assigned to such intent. This helps nudge the joint optimization process in MGDA to favor the low-latency intent more than others.

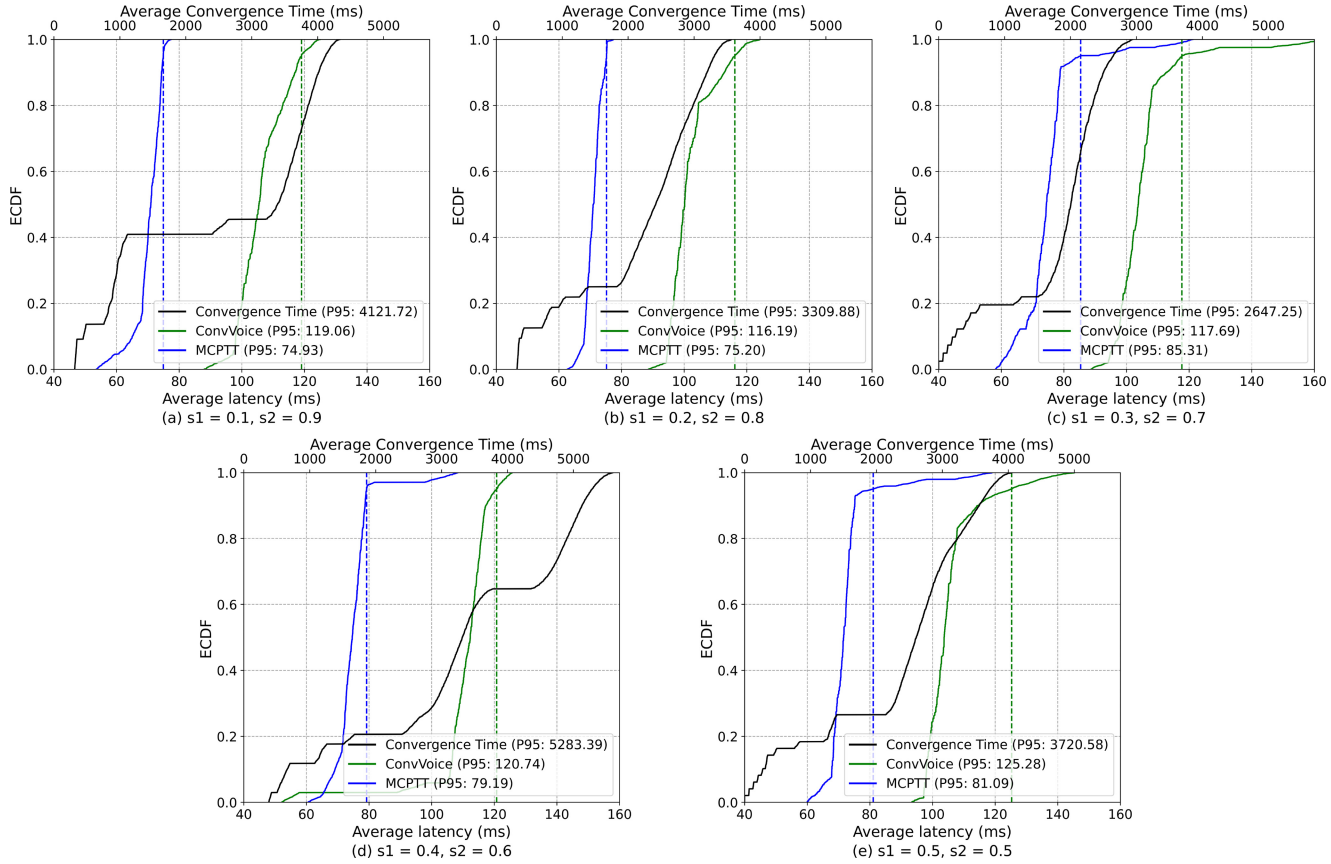


Fig. 9. Variance of SPSA & MGDA convergence time with given scaling factors (*s1, s2*) = (0.1,0.9), (0.2,0.8), (0.3,0.7), (0.4,0.6), (0.5,0.5) for ConvVoice and MCPTT intents.

Figure 9 depicts the ECDF for various combinations of scaling factors with a mapped *ConvVoice* and a *MCPTT* intent. The convergence time for given scaling factor values is calculated by estimating the time taken for one SPSA & MGDA iteration. The plots show the convergence behavior for distinct combinations of the scaling factor values (*s1,s2*) = [(0.1, 0.9), (0.2,0.8), (0.3,0.7), (0.4,0.6), (0.5,0.5)]. The plots also provide the achieved E2E latency for the deployed intents with highlighted 95% percentile values (P95) calculated for the independent simulation runs. We have identified two scaling factor combinations (Figure 9 (a, b)) based on the ECDFs for deployed intent latencies and convergence times.

In Figure 9(b), the scaling factors (*s1,s2*) = (0.2,0.8) produce the best values for the intent latencies and optimization convergence time (*MCPTT*: 75.00*ms*, *ConvVoice*: 116.58*ms*, Convergence: 3314.61*ms*). In Figure 9(a), the scaling factors (*s1,s2*) = (0.1,0.9) record the closest observed values for latencies, but worse convergence time. The rest of the combinations take too long to converge for the simulation runs. The observed results for the best scaling factors are in line with the prioritization model of the SL intents as defined by the cumulative loss function in Eq. (8). We utilize these scaling factors to ensure intent compliance for SL intents and minimal convergence time.

C. Scalability Analysis

This section explores varied intent deployment scenarios using the proposed algorithm. We demonstrate the impact of intent scaling on the observed performance of strict latency (SL) and non-strict latency (NSL) intents. These categories are defined according to the expected latency targets of the intents. Intents with a target latency lower than $50ms$ are considered as SL, and intents with a target latency greater than $50ms$ are NSL.

The number of SL and NSL intents increases geometrically as 2, 4, 8, 16, 32, and 64 with an equal distribution of SL and NSL intents to test the scalability of the proposed optimization framework. Distinct combinations of SL and NSL intents are generated and deployed in independent runs to test the resilience of the optimization algorithm. For example, for 4 intents, 2 SL and 2 NSL intents are deployed as a combination. All such combinations are deployed, and the proposed intent optimization is performed for 20 independent runs for each combination. A sample intent combination that can be deployed is given as, {(Smart Grid, Vehicular), (ConvVideo, MCPTT)}. It should be noted that the required resources are scaled as per the defined ranges for different intent sizes and combinations as shown in Table III.

The following subsections analyze the convergence of the proposed Algorithm 2 for different numbers of deployed

TABLE III
SIMULATION PARAMETERS

Parameter	Value
No. gNB sites	4
Sectors per site	3
No. of UEs	30
Tx Power	43 dBm
Path loss model	3GPPPropagationLossModel
Mobility model	
- NSL intents	ConstantPosition
- SL intents	RandomWaypointMobilityModel
Scheduler	Proportional fair
Shadow Fading	Log-normal, std=8 dB
Adaptive Modulation and Coding model	PiroEW2010
Cell layout radius	500 m
Carrier Bandwidth	[20, 40, 100] MHz
Traffic Shaping	FQ-CoDel
Traffic Model	Poisson Point Process
Numerology	
- NSL intents	[2, 3]
- SL intents	[0]
MGDA scaling factors	
- NSL intents (s1)	[0.1, 0.2, 0.3, 0.4, 0.5]
- SL intents (s2)	[0.9, 0.8, 0.7, 0.6, 0.5]
Best scaling factor(s1, s2)	(0.2, 0.8)
Confidence level	95%
No. of independent runs	20
No. of deployed intents	[2, 4, 8, 16, 32, 64]
No. of intent combinations	[30, 150, 75, 150, 75, 150]
Latency Targets (ms)	
- NSL Intents	[60, 75, 100, 100, 100, 200]
- SL Intents	[5, 10, 10, 30, 50]
Intent Types	
- NSL Intents	[MC-Signaling, MCPTT, ConvVideo, ConvVoice, Voice Video Gaming, MC-Data]
- SL Intents	[Smart Grid, Industry Automation, Low-Latency Broadband, Vehicular, Interactive Gaming]

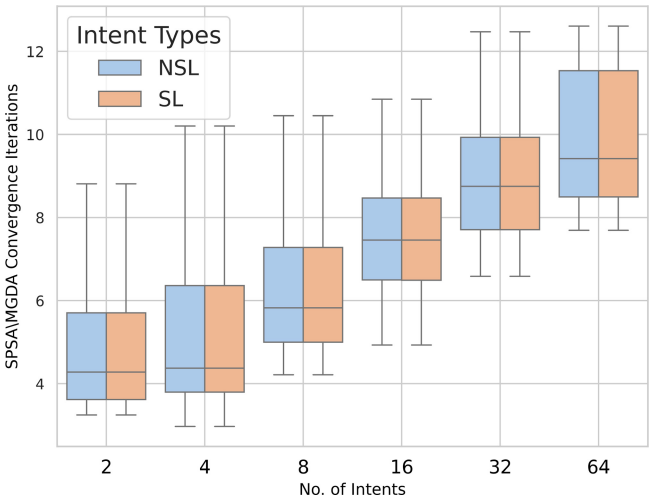


Fig. 10. Algorithm 2 convergence when scaling number of deployed intents.

intents, as well as the observed latency and the expected latency for SL and NSL. We also consider *best effort* and *random* deployment as baselines for comparison with the optimized *QoS*-based intent deployment solution proposed in this paper.

1) Convergence Performance: The performance of Algorithm 2 is evaluated for different number of intents. We observe its performance by assessing the number of iterations for convergence with objectives as shown in Figure 10. As expected, we observe an increasing trend proportional to the increase in the number of objectives (intents) optimized by the algorithm. However, the increase is not geometric, which is true for the number of intents. For example, the median number of iterations required for convergence for 64 intents is only twice that required for 2 intents deployed in the same system. It is observed that the convergence performance of SL and NSL intents is similar for different numbers of deployed intents. This is attributed to the mutual optimization nature of MGDA in Algorithm 2. All the objectives (or deployed intents) are optimized simultaneously, and a solution satisfying these objectives is determined by MGDA, resulting in the simultaneous convergence of all the deployed intents. In addition, the use of scaling factor in favor of the lowest latency SL intents skews the proposed MGDA algorithm towards them, resulting in improved compliance in comparison to the NSL intents.

2) Metric Performance: We compare the observed latency for SL and NSL intents with their respective latency targets by scaling the number of simultaneous deployed intents to assess the robustness of the proposed optimization process as shown in Figure 11 and Figure 12, respectively. Here, we consider the QoS-based intent deployment strategy, which represents the consideration of latency values for allocation of resources for deployed intents. The observed latencies are the cumulative average of all combinations for the given number of deployed intents, with 20 independent runs each.

In Figure 11, the observed latency increases with the simultaneous deployment of an increasing number of intents for SL intents using the QoS-based deployment strategy. This increase can be explained using the intuition behind the combined SPSA and MGDA algorithm, where increasing optimization objectives must be resolved for more intents. This trend is common for considered SL intents with an increased variance for interactive gaming and vehicular intent types. The increased variance is attributed to the division criterion between SL (target latency $\leq 50ms$) and NSL (target latency $> 50ms$) intents. The compliance of SL intents with their target latencies increases as the latency targets are relaxed. For example, for *Smart Grid* (target: $5ms$) and *Industry Automation* (target: $10ms$), the QoS-based strategy is fully compliant when the number of intents is 2, 4, 8, 16, 32 and approximately only 75% or greater for 64 intents. Whereas, *Vehicular* (target: $30ms$) and *Interactive Gaming* (target: $50ms$) are compliant when the number of intents is 2, 4, 8, and 16, whereas and only 70% or greater for 32 and 64 intents. This is attributed to the allocated scaling factors for low-latency intents, which are inversely proportional to the target latency. This implies that the intents with the lowest target latency will be optimized more aggressively in a combined deployment scenario with SL and NSL intents.

In Figure 12, the compliance of NSL intents using the QoS-based intent deployment strategy depicts that scaling the number of deployed intents degrades the observed latency compared to the target latency. The observed latency for NSL intents increases as the number of intents increases using dynamic allocation of resources. The variance for the observed latency is higher for a larger number of intents due to the preferred optimization of low-latency SL intents, sacrificing the required resources for the NSL intents. Similarly, the compliance ratio for NSL intents decreases as the latency

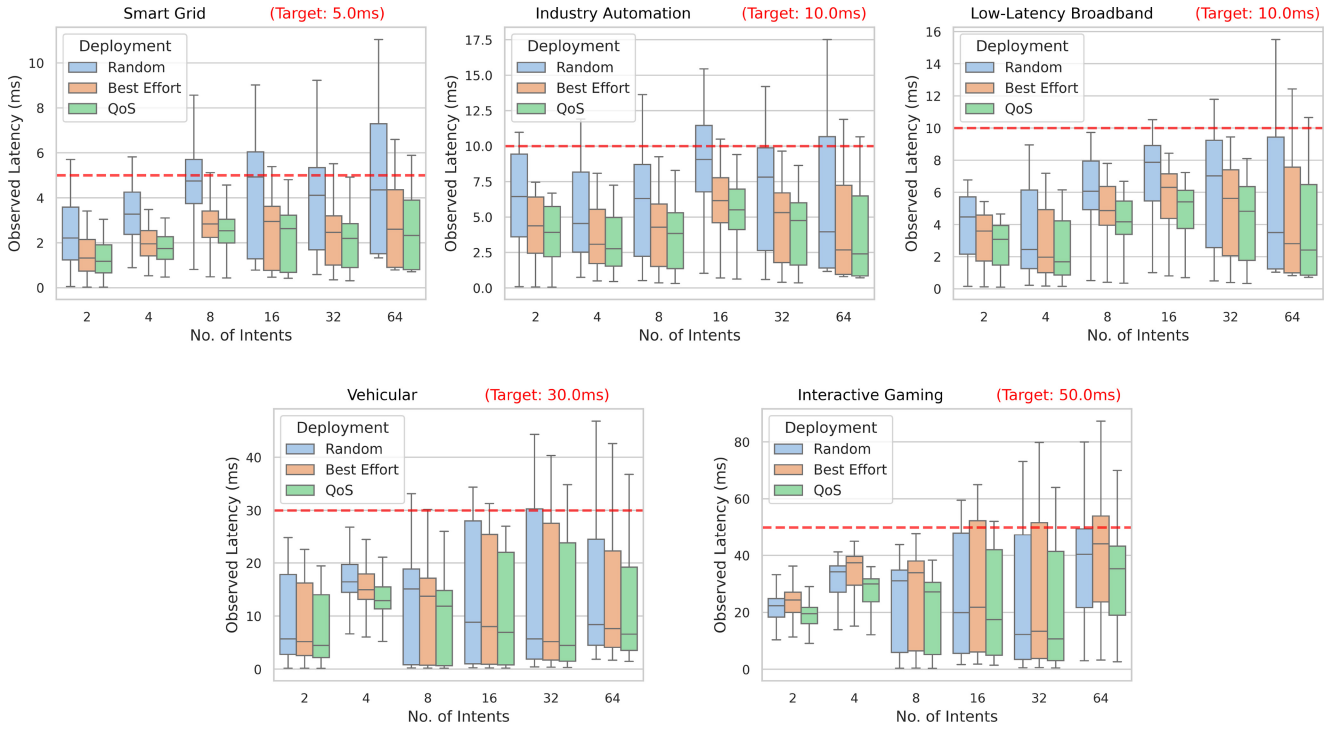


Fig. 11. SL intent performance for various deployment scenarios and number of intents. *SL intents*: [Smart Grid, Industry Automation, Low-Latency Broadband, Vehicular, Interactive Gaming], *number of deployed intents*: [2, 4, 8, 16, 32, 64].

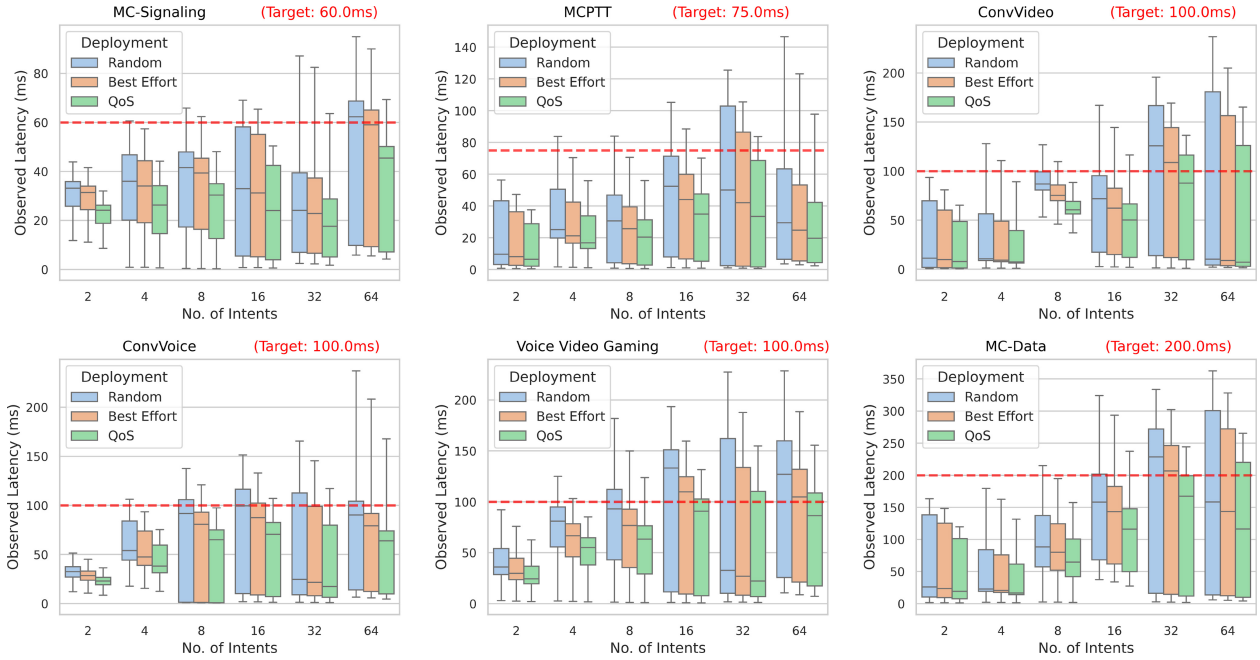


Fig. 12. NSL intent performance for various deployment scenarios and number of intents. *NSL intents*: [MC-Signaling, MCPTT, ConvVideo, ConvVoice, Voice Video Gaming, MC-Data], *number of deployed intents*: [2, 4, 8, 16, 32, 64].

targets increase. For example, *MC-Signaling* (target: 60*ms*) and *MCPTT* (target: 75*ms*) are fully compliant for 2, 4, 8, and 16 intents and approximately 70% or greater compliance for 32 and 64 intents. In contrast, *ConvVoice* (target: 100*ms*) and *Voice Video Gaming* (target: 100*ms*) are fully compliant for only 2 and 4 intents, whereas approximately 70% or less for 8, 16, 32, and 64 intents.

3) Baseline Comparison: In this section, we compare the performance of the proposed optimization algorithm with baseline intent deployment scenarios. We have the following three scenarios: 1) **Best effort**: the resources are allocated on a first-come first-serve basis and without any specific guarantees for any particular service intent, 2) **Random**: the resources are allocated randomly without any specific criterion, and

3) **QoS**: the resources are allocated according to the 5GQI specifications [42, Table V.7.4-1] for 5G service intents. The number of intents is 2, 4, 8, 16, 32, and 64. Here, best effort and random scenarios act as simple baselines for comparison, and the results obtained are discussed as follows.

Figure 11 identifies the observed latency for the SL intents as per deployment strategy. *Random* baseline allocates resources to the respective intents without considering their target latencies. This results in the worst performance among the available strategies with the lowest percentage of compliance across all numbers of deployed intents. Stringent latency requirements (like *Smart Grid* and *Industry Automation*) offer the lowest compliance (often $\leq$60%) with high variance for all numbers of deployed intents using *best effort* and *random* baseline strategies. *Best effort* baseline provides improved compliance than *random* baseline, which is attributed to the deterministic deployment of intents and the ability of the optimization algorithm to prefer low-latency intents. The compliance ratio of SL intents is higher for *best effort* and *random* baselines for a smaller number of deployed intents. However, as the intent number increases, the compliance ratio decreases significantly compared to the *QoS*-based intent deployment. *QoS*-based strategy consistently outperforms both baselines, maintaining the highest percentage of observed latencies below the target with the narrowest distribution, demonstrating its effectiveness in providing reliable low-latency performance even under high load for all SL intent types.

Figure 12 shows the observed latency for NSL intents for *best effort*, *random*, and *QoS*-based intent deployment. The results indicate that *random* baseline leads to the highest variance in the observed latency and inconsistent performance across all numbers of intents. In contrast, *best effort* baseline improves latency compliance, but it does not fully align with the target latencies for all the number of intents. *QoS*-based deployment stands out by consistently delivering the lowest latencies and the most compact distribution across all NSL intents and number of intents, ensuring the highest level of compliance. In addition, the proposed *QoS*-based deployment strategy for NSL intents records a lower compliance when compared with the *QoS*-based deployment of SL intents. This shows that the MGDA scaling factor normalized using the target latencies of relevant intents plays a role as long as the same guarantees are possible in the underlying deployed network infrastructure.

VII. Conclusion

This paper proposes an intent processing pipeline based on knowledge extraction and utilization for service, such as intent mapping, translation, validation, and conflict management. This is achieved by representing the available context information from network stakeholders in a service orchestration framework. The available data is numerically expressed in a knowledge embedding domain to enable the provisioning of inference and validation processes in the intent processing pipeline. Furthermore, the intent deployment is achieved via a simulated 5G network environment with services deployed against translated intents. We perform extensive simulation experiments for the intent translation and optimized deployment of natural languages (NLs), producing promising results for the considered service use cases. It shows that KG-based processing and network context management is a potential enabler for IBN and autonomous networks.

We also intend to explore and enhance our custom KG to include a scaled version of the network management ontology to perform contextual decisions in the network domains. This will aid us in understanding the explicit impact of incorporating IBN with traditional network management concepts such as policy-driven networks.

References

[1] "Intent in autonomous networks," TMF, Amsterdam, The Netherlands, Introductory Document IG1253, 2022. [Online]. Available: https://www.tmforum.org/resources/how-to-guide/ig1253-intent-in-autonomous-networks-v1-3-0/

[2] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Intent-based networking—Concepts and definitions," Internet Eng. Task Force, RFC 9315, Oct. 2022. [Online]. Available: https://www.rfc-editor.org/info/rfc9315

[3] A. Leivadeas and M. Falkner, "A survey on intent-based networking," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 625–655, 1st Quart., 2023.

[4] X. Chang et al., "KID: Knowledge graph-enabled intent-driven network with digital twin," in *Proc. 27th Asia Pacific Conf. Commun. (APCC)*, 2022, pp. 272–277.

[5] M. Avgeris, A. Leivadeas, N. Athanasopoulos, I. Lambadaris, and M. Falkner, "Model predictive control for automated network assurance in intent-based networking enabled service function chains," in *Proc. IEEE/IFIP Netw. Oper. Manag. Symp.*, Miami, FL, USA, May 2023, pp. 1–7. [Online]. Available: https://ieeexplore.ieee.org/document/10154316/

[6] K. Mehmood, K. Kralevska, and D. Palma, "Knowledge-based intent modeling for next generation cellular networks," in *Proc. IEEE Int. Mediterr. Conf. Commun. Netw. (MeditCom)*, 2023, pp. 181–186.

[7] A. S. Jacobs, R. J. Pfitscher, R. A. Ferreira, and L. Z. Granville, "Refining network intents for self-driving networks," in *Proc. Afternoon Workshop Self-Driv. Netw.*, 2018, pp. 15–21. [Online]. Available: https://doi.org/10.1145/3229584.3229590

[8] (World Wide Web Consortium (W3C), Cambridge, MA, USA). *Owl 2 Web Ontology Language; Structural Specification and Functional-Style Syntax*. 2012. [Online]. Available: https://www.w3.org/TR/owl2-syntax/

[9] (World Wide Web Consortium (W3C), Cambridge, MA, USA). *RDF 1.1 Primer*. 2014. [Online]. Available: https://www.w3.org/TR/2014/NOTE-rdf11-primer-20140624/

[10] "Intent common model v2.1.0," TMF, Amsterdam, The Netherlands, Rep. TR292, 2022. [Online]. Available: https://www.tmforum.org/resources/reference/tr292-intent-management-ontology-v1-1-0/

[11] K. Mehmood, K. Kralevska, and D. Palma, "Knowledge graph embedding in intent-based networking," in *Proc. IEEE 10th Int. Conf. Netw. Softw. (NetSoft)*, 2024, pp. 13–18.

[12] J. Spall, "Implementation of the simultaneous perturbation algorithm for stochastic optimization," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 34, no. 3, pp. 817–823, Jul. 1998.

[13] J.-A. Désidéri, "Multiple-gradient descent algorithm (MGDA) for multiobjective optimization," *Comptes Rendus Mathematique*, vol. 350, no. 5, pp. 313–318, 2012. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1631073X12000738

[14] H. Yu, H. Rahimi, C. Janz, D. Wang, C. Yang, and Y. Zhao, "A comprehensive framework for intent-based networking, standards-based and open-source," in *Proc. IEEE/IFIP Netw. Oper. Manag. Symp.*, 2023, pp. 1–6.

[15] A. Clemm and T. Eckert, "Combining autonomic and intent-based networking," in *Proc. IEEE/IFIP Netw. Oper. Manag. Symp.*, 2023, pp. 1–6.

[16] M. Gharbaoui, B. Martini, and P. Castoldi, "Intent-based networking: Current advances, open challenges, and future directions," in *Proc. 23rd Int. Conf. Transp. Opt. Netw. (ICTON)*, 2023, pp. 1–5.

[17] S. Barzegar, M. Ruiz, and L. Velasco, "Intent-based networking for zero-touch optical networking," in *Proc. 23rd Int. Conf. Transp. Opt. Netw. (ICTON)*, 2023, pp. 1–4.

[18] A. Leivadeas and M. Falkner, "Autonomous network assurance in intent based networking: Vision and challenges," in *Proc. 32nd Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2023, pp. 1–10.

[19] J. Zhang et al., "Intent-driven closed-loop control and management framework for 6G open RAN," *IEEE Internet Things J.*, vol. 11, no. 4, pp. 6314–6327, Feb. 2024.
[20] P. Lingga, J. J. Kim, and J. Jeong, "Intent-based network management in 6G core networks," in *Proc. 13th Int. Conf. Inf. Commun. Technol. Converg. (ICTC)*, 2022, pp. 760–762.
[21] J. Massa, S. Forti, F. Paganelli, P. Dazzi, and A. Brogi, "Declarative provisioning of virtual network function chains in intent-based networks," in *Proc. IEEE 9th Int. Conf. Netw. Softw. (NetSoft)*, 2023, pp. 522–527.
[22] Y. Abassi and N. Tabbane, "Context aware life cycle for intent-based networking systems," in *Proc. IEEE 10th Int. Conf. Commun. Netw. (ComNet)*, 2023, pp. 1–8.
[23] C. Yang, X. Mi, Y. Ouyang, R. Dong, J. Guo, and M. Guizani, "SMART intent-driven network management," *IEEE Commun. Mag.*, vol. 61, no. 1, pp. 106–112, Jan. 2023.
[24] F. J. Z. Mier, C. A. Iglesias, and A. Sheth, "Applications of knowledge graphs in Telecommun. Syst. management," *IEEE Internet Comput.*, vol. 27, no. 3, pp. 29–34, May/Jun. 2023.
[25] L. Jie, Z. Feng, M. Zhang, F. Jing, and Q. Guo, "Review of knowledge graph and its vertical applications in industry," in *Proc. 42nd Chinese Control Conf. (CCC)*, 2023, pp. 5151–5157.
[26] A. Mitropoulou, P. Kokkinos, and E. Varvarigos, "Identifying network congestion using knowledge graphs and link prediction," in *Proc. IEEE/ACM 16th Int. Conf. Util. Cloud Comput.*, 2024, pp. 1–6. [Online]. Available: https://doi.org/10.1145/3603166.3632129
[27] S. He, Y. Ou, L. Wang, H. Zhan, P. Ren, and Y. Huang, "Representation learning of knowledge graph for wireless communication networks," 2022, *arXiv:2208.10496*.
[28] K. Mitropoulou, P. Kokkinos, P. Soumplis, and E. Varvarigos, "Anomaly detection in cloud computing using knowledge graph embedding and machine learning mechanisms," *J. Grid Comput.*, vol. 22, no. 1, p. 6, Dec. 2023. [Online]. Available: https://doi.org/10.1007
[29] "Intent management ontology v1.1.0," TMF, Amsterdam, The Netherlands, Rep. TR290, 2022. [Online]. Available: https://www.tmforum.org/resources/technical-report/tr290-intent-common-model-v2-1-0/
[30] D. Daroui, M. D'Angelo, L. Mokrushin, M. Orlić, and A. C. Baktir, "State management of knowledge base in intent management functions in 6G networks," in *Proc. Int. Balkan Conf. Commun. Netw. (BalkanCom)*, 2023, pp. 1–6.
[31] J. Zhang et al., "A conflict resolution scheme in intent-driven network," in *Proc. IEEE/CIC Int. Conf. Commun. China (ICCC)*, 2021, pp. 23–28.
[32] Y. Xiao, W. Quan, H. Zhou, M. Liu, and K. Liu, "Lightweight natural language driven intent translation mechanism for intent based networking," in *Proc. 7th Int. Conf. Comput. Commun. Syst. (ICCCS)*, 2022, pp. 46–51.
[33] D. Wang, S. Zhang, R. Su, H. Li, and X. Xia, "An intent-based network empowered by knowledge graph: Enhancement of intent translation and management function for vertical industry," in *Proc. IEEE/CIC Int. Conf. Commun. China (ICCC Workshops)*, 2023, pp. 1–6.
[34] Z. Guo, F. Li, J. Shen, T. Xie, S. Jiang, and X. Wang, "ConfigReco: Network configuration recommendation with graph neural networks," *IEEE Netw.*, vol. 38, no. 1, pp. 7–14, Jan. 2024.
[35] S. Kou, C. Yang, and M. Wu, "SAFLA: Semantic-aware full lifecycle assurance designed for intent-driven networks," 2024, *arXiv:2404.12305*.
[36] A. Boutouchent et al., "AMANOS: An intent-driven management and orchestration system for next-generation cloud-native networks," *IEEE Commun. Mag.*, vol. 62, no. 6, pp. 42–49, Jun. 2024.
[37] "Principles for a telecommunications management network," ITU-Rec. M.3010, Int. Telecommun. Union, Geneva, Switzerland. Accessed: Apr. 25, 2023. [Online]. Available: https://www.itu.int/rec/T-REC-M.3010-199210-S/en
[38] S. Sabour, A. Ebrahimzadeh, F. Wuhib, M. Soualhia, and R. H. Glitho, "Service graphs generation in intent-based networks," in *Proc. IEEE 21st Consum. Commun. Netw. Conf. (CCNC)*, 2024, pp. 90–97.
[39] A. Randles, D. O'Sullivan, J. Keeney, and L. Fallon, "Ontology driven closed control loop automation," in *Proc. IEEE 9th Int. Conf. Netw. Softw. (NetSoft)*, 2023, pp. 528–533.
[40] Y. Ouyang, F. Li, C. Yang, R. Song, X. Liu, and Z. Ji, "Ontology-based network intent refinement framework," in *Proc. IEEE 22nd Int. Conf. Commun. Technol. (ICCT)*, 2022, pp. 1394–1399.
[41] N. Dragun and N. Bogunović, "Enhancing intent-driven networking with granular and aspect approach," in *Proc. 33rd Int. Telecommun. Netw. Appl. Conf.*, 2023, pp. 271–276.
[42] *5G; System Architecture for the 5G System (5Gs)*, 3GPP Standard TS 23.501, 2022. [Online]. Available: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3144
[43] *Management and Orchestration; Intent Driven Management Services for Mobile Networks*, 3GPP Standard TS 28.312, 2022. [Online]. Available: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3554
[44] S. He, K. Liu, G. Ji, and J. Zhao, "Learning to represent knowledge graphs with gaussian embedding," in *Proc. 24th ACM Int. Conf. Inf. Knowl. Manag.*, 2015, pp. 623–632. [Online]. Available: https://doi.org/10.1145/2806416.2806502
[45] G. Ji, S. He, L. Xu, K. Liu, and J. Zhao, "Knowledge graph embedding via dynamic mapping matrix," in *Proc. 53rd Annu. Meeting Assoc. Comput. Linguist. 7th Int. Joint Conf. Nat. Lang. Process.*, Jul. 2015, pp. 687–696. [Online]. Available: https://aclanthology.org/P15-1067
[46] Z. Wang, J. Zhang, J. Feng, and Z. Chen, "Knowledge graph embedding by translating on hyperplanes," in *Proc. AAAI Conf. Artif. Intell.*, vol. 28, Jun. 2014, pp. 1112–1119. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/8870
[47] T. Trouillon, J. Welbl, S. Riedel, E. Gaussier, and G. Bouchard, "Complex embeddings for simple link prediction," in *Proc. 33rd Int. Conf. Mach. Learn.*, vol. 48, Jun. 2016, pp. 2071–2080. [Online]. Available: https://proceedings.mlr.press/v48/trouillon16.html
[48] J. Spall, "Accelerated second-order stochastic optimization using only function measurements," in *Proc. 36th IEEE Conf. Decision Control*, vol. 2, 1997, pp. 1417–1424.
[49] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 2, pp. 494–514, Feb. 2022.



Kashif Mehmood (Graduate Student Member, IEEE) received the B.Sc. degree in telecommunications engineering from NUCES-FAST, Islamabad, Pakistan, the M.Eng. degree in information and communication systems from Sejong University, Seoul, South Korea, and the Ph.D. degree from the Norwegian University of Science and Technology (NTNU), Norway, in 2025, with a focus on intent modeling, secure orchestration, knowledge graph applications, conflict resolution, game theory, and model/policy-based management. With research experience at Sejong University and various communication service providers, his work centers on autonomous management of next-generation cellular networks through intent-based networking and knowledge engineering. He also serves as a reviewer for different Elsevier, IEEE, and Wiley journals.



Katina Kralevska (Member, IEEE) received the M.Sc. degree in wireless and mobile communications from the Institute of Telecommunications, Ss. Cyril and Methodis University, Skopje, North Macedonia, in 2012, and the Ph.D. degree in telematics from the Norwegian University of Science and Technology, Trondheim, Norway, in 2016, where she is an Associate Professor with the Department of Information Security and Communication Technology. She was the Deputy Head of Department from 2019 to 2020 and a Co-Founder of MemoScale, a company for data storage and compression. Her research interests and activities lie in the areas of network virtualization and management, coding theory, and blockchain.



David Palma received the B.Sc. degree in informatics engineering and the Ph.D. degree in information science and technology from the University of Coimbra, Portugal, in 2007 and 2013, respectively. He is an Associate Professor with the Department of Information Security and Communication Technology, Norwegian University of Science and Technology (NTNU), Trondheim, Norway. He was an H2020 Marie Sklodowska-Curie Postdoctoral Fellow with NTNU and has worked in the past as a Researcher and a Project Manager, as well as an invited Assistant Professor with the University of Coimbra. He has authored and coauthored multiple papers in refereed conferences and journals. His current research interests are on autonomic computer networks and knowledge management. He is actively involved in the research community, serving on technical program committees and as a reviewer for international journals and conferences. He has also contributed to the preparation of several successful national and European research proposals.